



Computer Vision for Autonomous Vehicles

Week 3: Image Formation and Visual Features

Instructor: Golnaz Habibi, PhD

University of Oklahoma

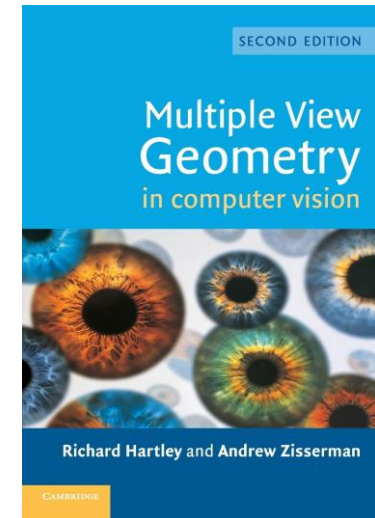
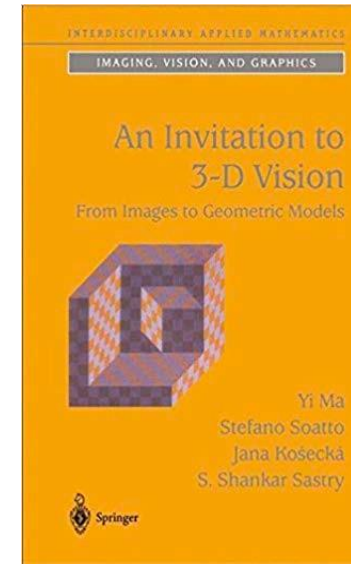
Fall 2025

September 9th, 2025



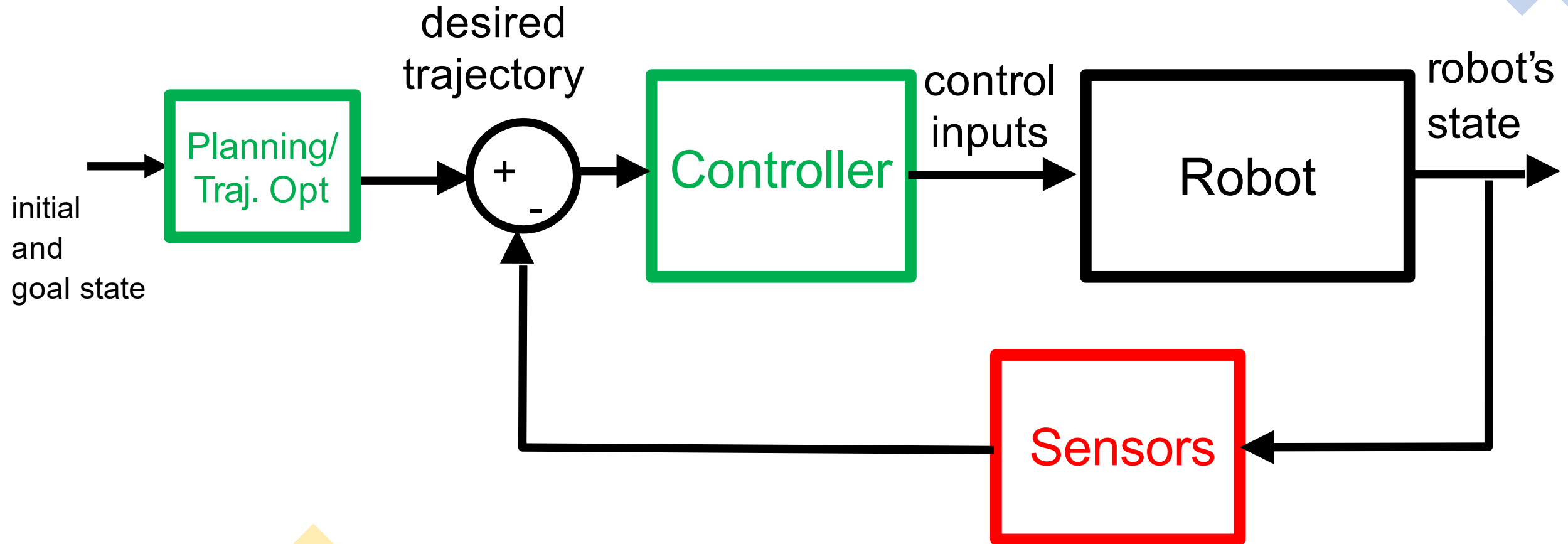
Announcement

- Lab 2 deadline is due this Friday Sept 12
- We plan to release robots this week!
- Today we start computer vision!
 - Image Formation
 - Pinhole Camera Model
 - We also review lab 2

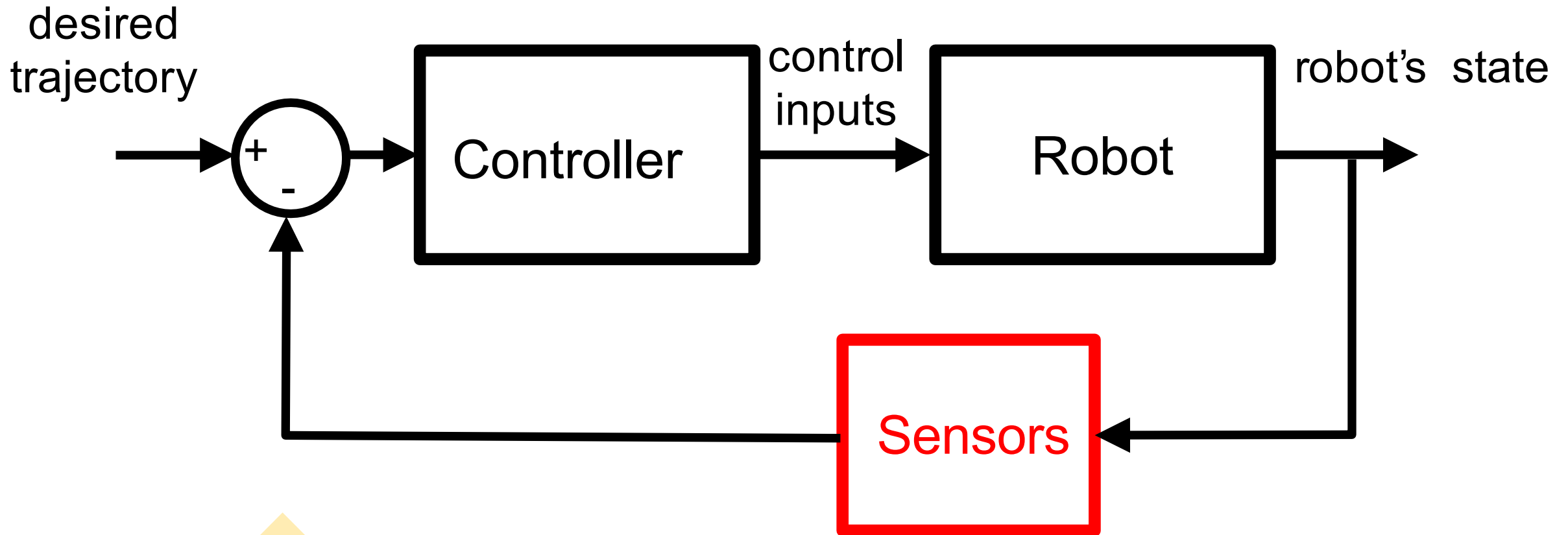


Chapter 3 Image Formation

Autonomous Navigation pipeline



What's next?



What's next?

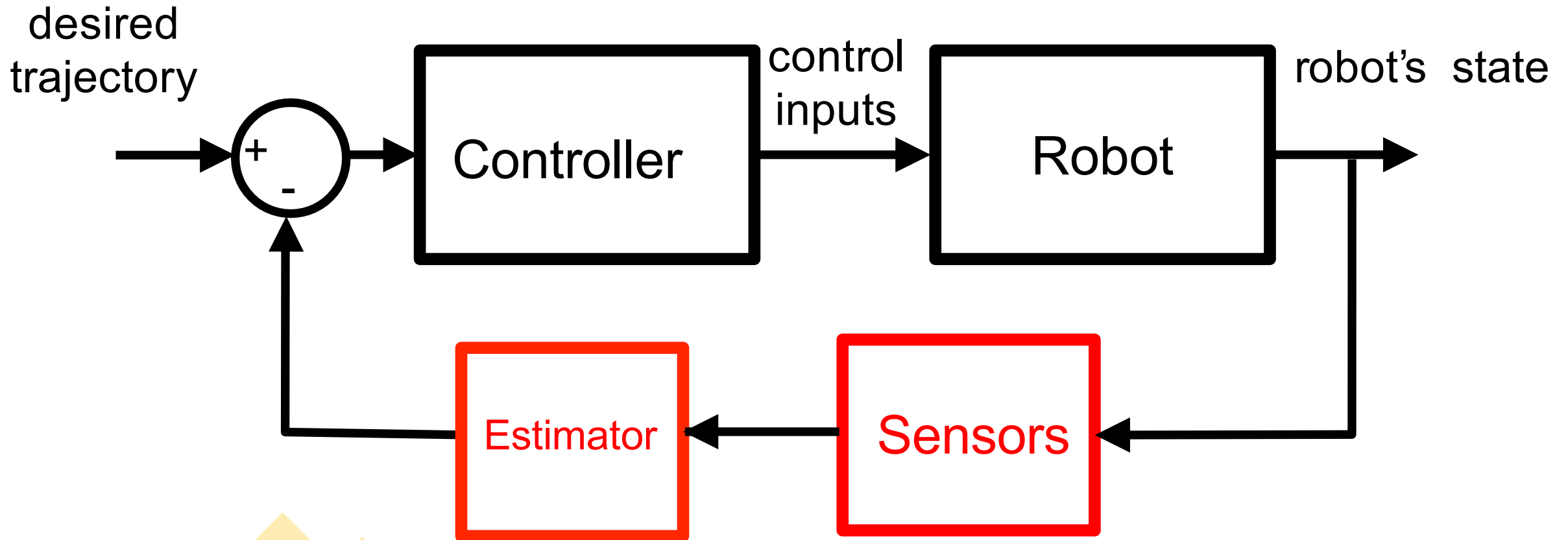
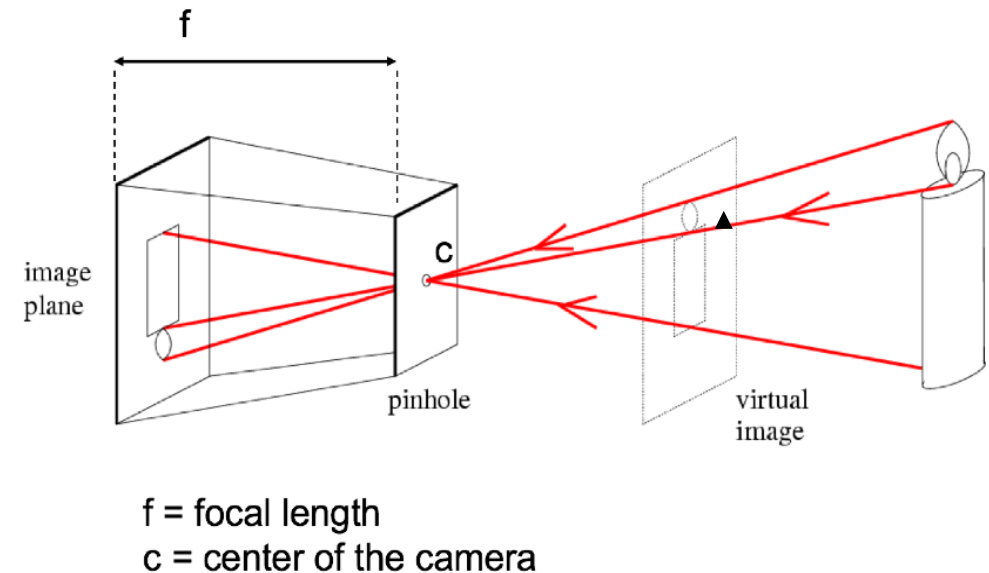
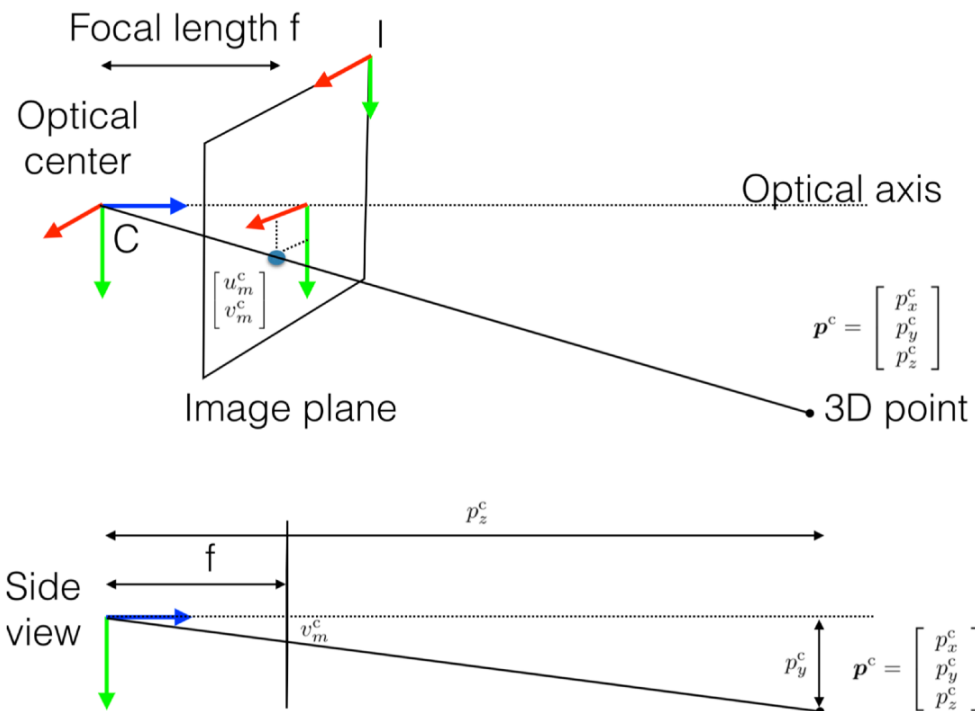


Image Formation and Pinhole Camera Model

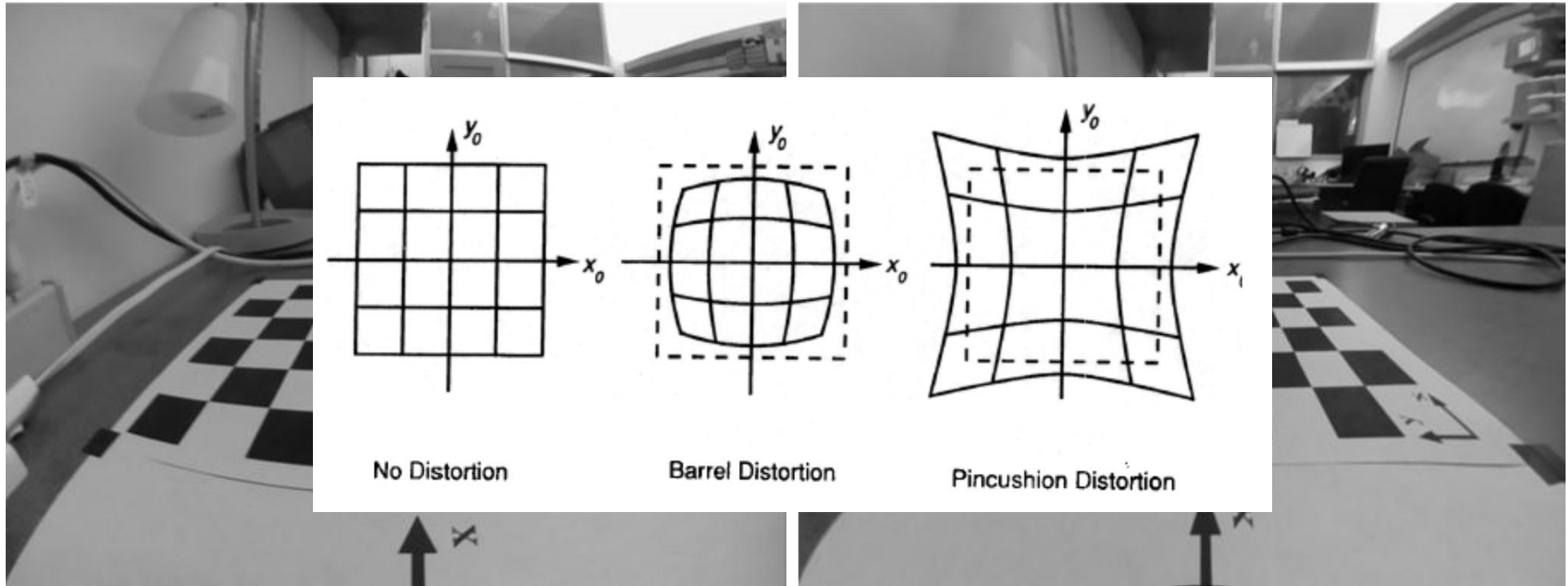
- How to capture a 3D scene on a 2D image?
- How to compute the 2D projection (pixel) of a given 3D point?



Frontal pinhole model

Adding Lenses...

- Pinhole model is based on the geometry of the **camera obscura**
- In practice: add a **lens** in front of the aperture to capture more light
- Pinhole model holds, but **distortion** may appear due lens imperfections



- distortion can be described mathematically using **distortion parameters**
- can be estimated during calibration and compensated for (**undistortion**)

Let's do some math!



Consider a 3D point $\mathbf{p}^c$ as shown in Fig. 11.1.

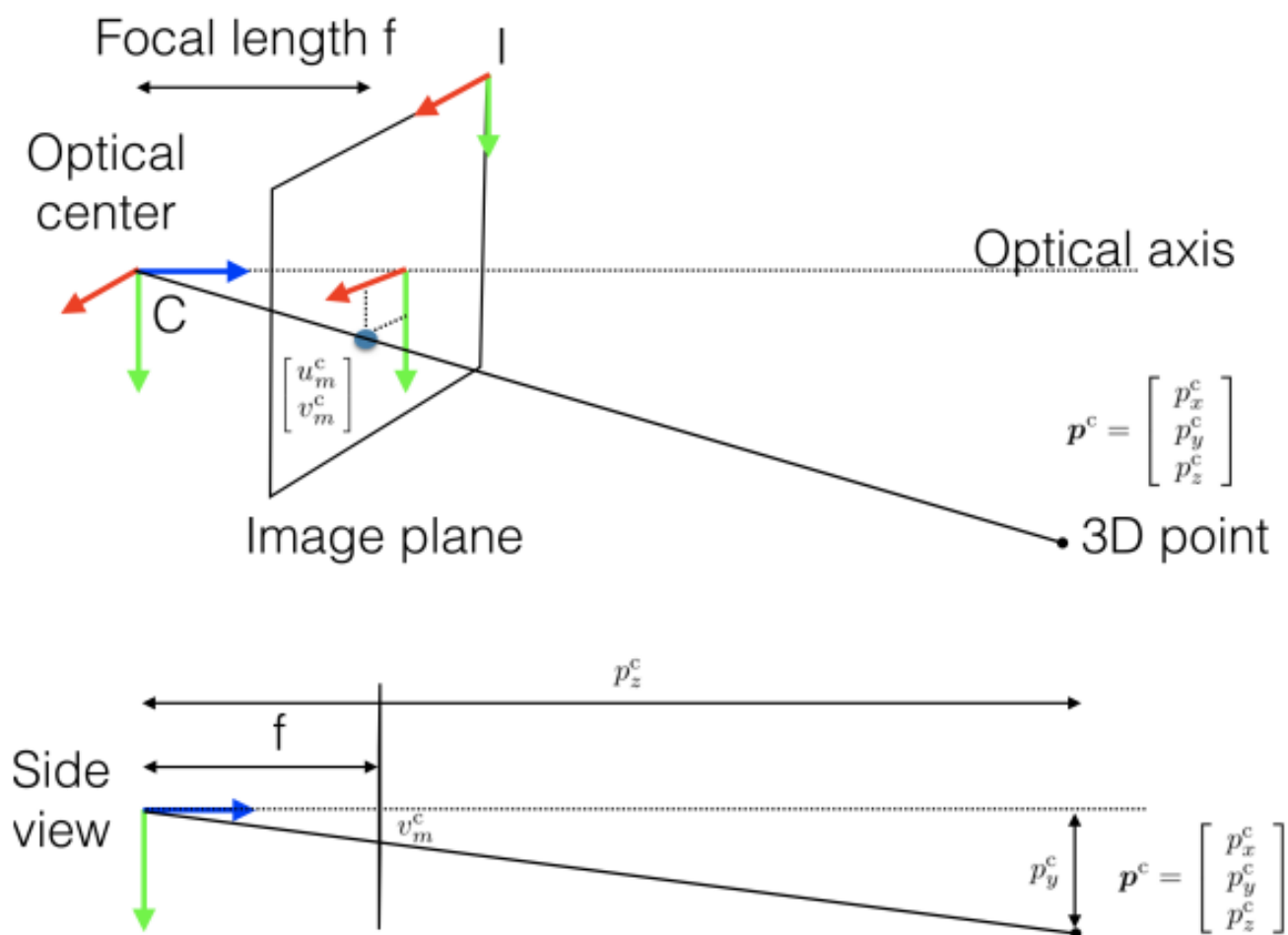


Figure 11.1: Pinhole Model.

Consider a 3D point $\mathbf{p}^c$ as shown in Fig. 11.1.

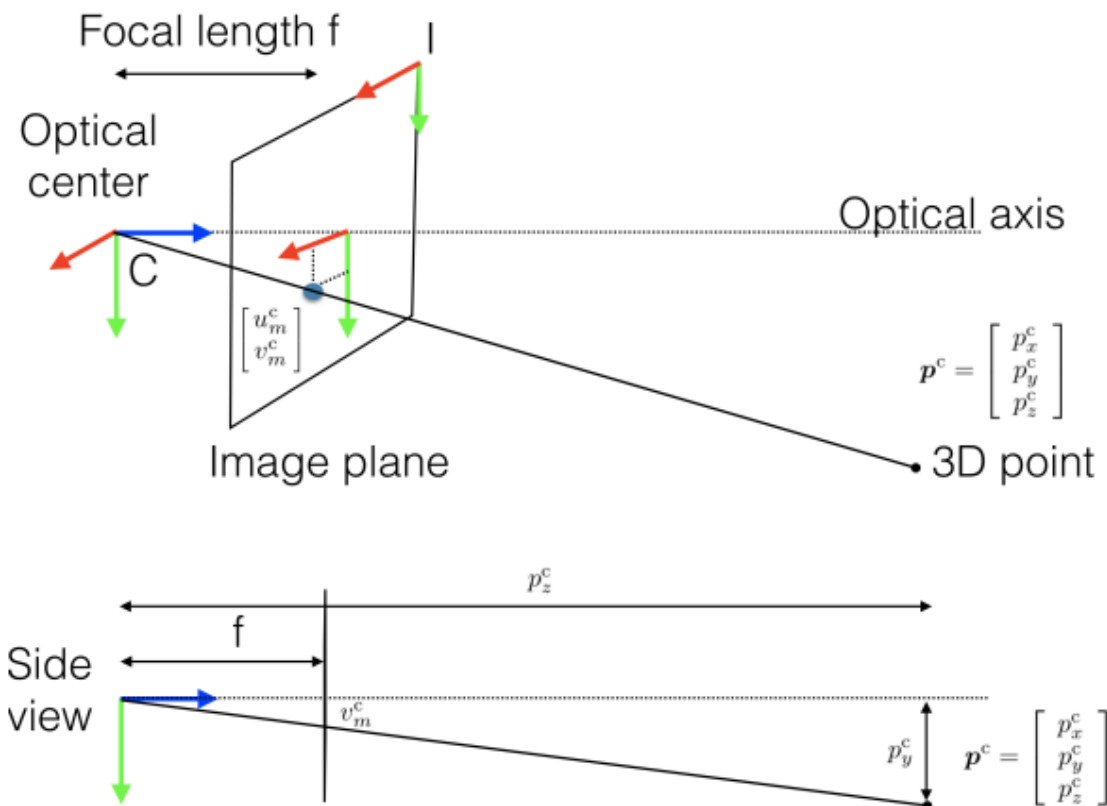


Figure 11.1: Pinhole Model.

The position (u_m^c, v_m^c) of the projection of $\mathbf{p}^c$ on the camera plane is:

$$u_m^c = f \frac{p_x^c}{p_z^c} \quad v_m^c = f \frac{p_y^c}{p_z^c}$$

where f is the *focal length* (in meters), and the subscript m denotes that (u_m^c, v_m^c) are still expressed in meters (later on we convert them into pixels).

The relations in (11.1) can be written, by moving p_z^c to the left-hand-side and using a matrix notation as:

$$p_z^c \begin{bmatrix} u_m^c \\ v_m^c \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x^c \\ p_y^c \\ p_z^c \end{bmatrix} \quad (11.2)$$

where the last equation simply states $p_z^c = p_z^c$. Rewriting $\mathbf{p}^c$ in homogeneous coordinates:

$$\mathbf{p}^c = \begin{bmatrix} p_x^c \\ p_y^c \\ p_z^c \end{bmatrix} \xrightarrow{\text{homogeneous coordinates}} \tilde{\mathbf{p}}^c = \begin{bmatrix} p_x^c \\ p_y^c \\ p_z^c \\ 1 \end{bmatrix} \quad (11.3)$$

which allows writing (11.2) as:

$$p_z^c \begin{bmatrix} u_m^c \\ v_m^c \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \tilde{\mathbf{p}}^c \quad (11.4)$$

which represents the canonical projection of a point $\tilde{\mathbf{p}}^c$ whose coordinates are given in the camera frame.

Conversion to pixels

Therefore, we can convert (u_m^c, v_m^c) to pixels and change the reference frame as follows:

$$u^I = s_x u_m^c + o_x \quad v^I = s_y v_m^c + o_y \quad (11.5)$$

Which using a compact matrix notation becomes:

$$\begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & o_x \\ 0 & s_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m^c \\ v_m^c \\ 1 \end{bmatrix} \quad (11.6)$$

If the pixels are skewed (non-rectangular), the expression can be generalized to:

$$\begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & s_\theta & o_x \\ 0 & s_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m^c \\ v_m^c \\ 1 \end{bmatrix} \quad (11.7)$$

Substituting (11.7) into (11.4):

$$p_z^c \begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & s_\theta & o_x \\ 0 & s_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \tilde{\mathbf{p}}^c \quad (11.8)$$

Substituting (11.7) into (11.4):

$$p_z^c \begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & s_\theta & o_x \\ 0 & s_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \tilde{\mathbf{p}}^c \quad (11.8)$$

Multiplying the first two matrices in the right-hand side:

$$p_z^c \begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} s_x & f & s_\theta & f & o_x \\ 0 & & s_y & f & o_y \\ 0 & & 0 & & 1 \end{bmatrix}}^K \overbrace{\begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix}}^{\Pi_0} \tilde{\mathbf{p}}^c \quad (11.9)$$

Terminology:

- $[o_x \ o_y]$ is called the *principal point*

- $s_x f$ is the focal length in horizontal pixels (s_x number of horizontal pixels per meter)
- $s_y f$ is the focal length in vertical pixels (s_y number of vertical pixels per meter)
- s_x/s_y is called the *aspect ratio*
- $s_\theta f$ is called the *skew* of the pixel
- $\mathbf{K}$ is called the *intrinsic (calibration) matrix*
- $\mathbf{\Pi}_0$ is called the *canonical projection*

11.1.2 Points coordinates in the world frame

When the point coordinates are given with respect to an external world frame w , i.e., $\mathbf{p}^w$, it is straightforward to adapt (11.9) as:

$$p_z^c \begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} s_x f & s_\theta f & o_x \\ 0 & s_y f & o_y \\ 0 & 0 & 1 \end{bmatrix}}^K \overbrace{\begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix}}^{\Pi_0} \mathbf{T}_w^c \tilde{\mathbf{p}}^w \quad (11.10)$$

Recalling the structure of the pose matrix $\mathbf{T}_w^c$ and multiplying by the canonical projection Π_0 :

$$p_z^c \begin{bmatrix} u^I \\ v^I \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} s_x f & s_\theta f & o_x \\ 0 & s_y f & o_y \\ 0 & 0 & 1 \end{bmatrix}}^{\text{intrinsic calibration}} \overbrace{\begin{bmatrix} \mathbf{R}_w^c & \mathbf{t}_w^c \end{bmatrix}}^{\text{extrinsic calibration}} \tilde{\mathbf{p}}^w = \Pi \tilde{\mathbf{p}}^w \quad (11.11)$$

Terminology:

- $\begin{bmatrix} \mathbf{R}_w^c & \mathbf{t}_w^c \end{bmatrix}$ is called the *extrinsic (calibration) matrix*
- Π is called the *projection matrix*

11.1.3 Projection in practice

In the following lectures, we will consider different uses of projective geometry. Here are two examples:

- computing pixel projection of a 3D point: if the pose of the camera $\mathbf{T}_w^c$ is known then we can project a 3D point $\tilde{\mathbf{p}}^w$ as $\mathbf{\Pi}\tilde{\mathbf{p}}^w$ and then retrieve the pixel projection of $\tilde{\mathbf{p}}^w$ as:

$$u^I = \frac{[\mathbf{\Pi}\tilde{\mathbf{p}}^w]_1}{[\mathbf{\Pi}\tilde{\mathbf{p}}^w]_3} \quad v^I = \frac{[\mathbf{\Pi}\tilde{\mathbf{p}}^w]_2}{[\mathbf{\Pi}\tilde{\mathbf{p}}^w]_3} \quad (11.12)$$

where $[\mathbf{\Pi}\tilde{\mathbf{p}}^w]_i$ denotes the i -th entry in $\mathbf{\Pi}\tilde{\mathbf{p}}^w \in \mathbb{R}^3$.

- if the pixel projection (u^I, v^I) and the corresponding 3D point are given, we can use the expression (11.11) to infer the pose of the camera (this requires multiple 3D points)

11.1.4 Distortion

Cameras with wide field of view are often subject to radial distortion, which can be modeled as:

$$u^c = (1 + a_1 r^2 + a_2 r^4) u_{distort}^c \quad v^c = (1 + a_1 r^2 + a_2 r^4) v_{distort}^c \quad (11.13)$$

where $r^2 = (u_{distort}^c)^2 + (v_{distort}^c)^2$ and a_i are the so-called *distortion coefficients*. Note that the equation above is expressed in the camera frame (origin at the center of the image, rather than on the top left) since the distortion increases with the distance from the center of the image. It is possible to rewrite the model using the image frame as:

$$u^I = (1 + a_1 r^2 + a_2 r^4)(u_{distort}^I - o_x) + o_x \quad v^I = (1 + a_1 r^2 + a_2 r^4)(v_{distort}^I - o_y) + o_y \quad (11.14)$$

and $r^2 = (u_{distort}^I - o_x)^2 + (v_{distort}^I - o_y)^2$. We typically assume the distortion coefficients to be known from previous calibration, hence we can always compensate for the calibration using the model (11.14).

Digital Images...

- 2D array of “light sensors”

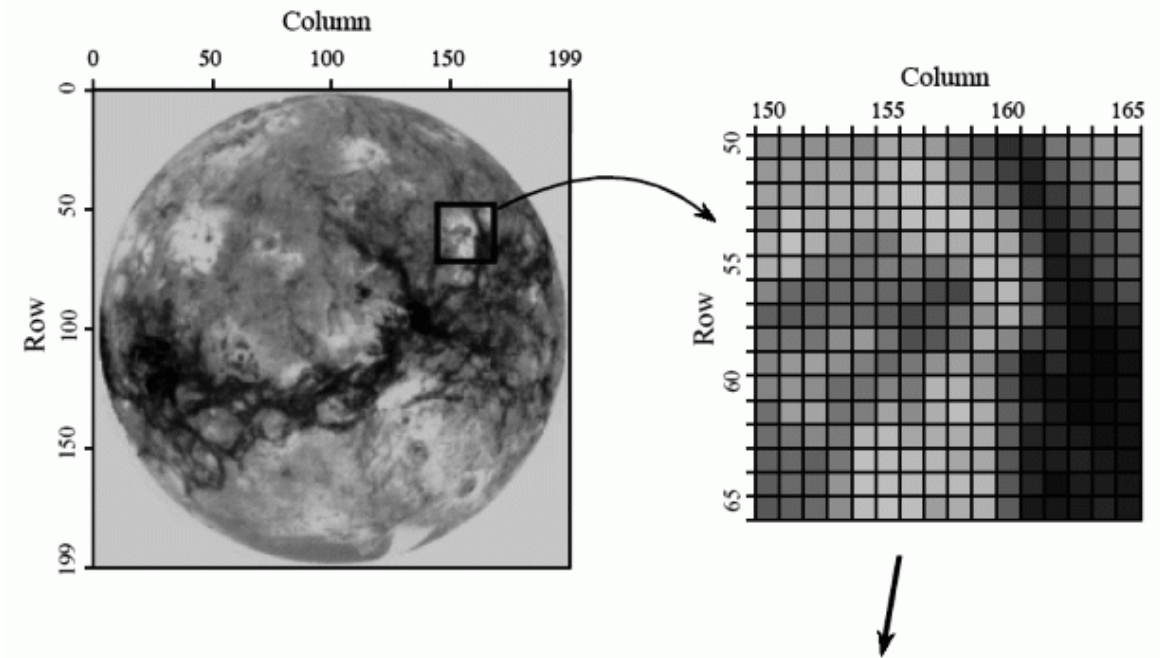


FIGURE 23-1
Digital image structure. This example image is the planet Venus, as viewed in reflected microwaves. Digital images are represented by a two-dimensional array of numbers, each called a *pixel*. In this image, the array is 200 rows by 200 columns, with each pixel a number between 0 to 255. When this image was acquired, the value of each pixel corresponded to the level of reflected microwave energy. A *grayscale* image is formed by assigning each of the 0 to 255 values to varying shades of gray.

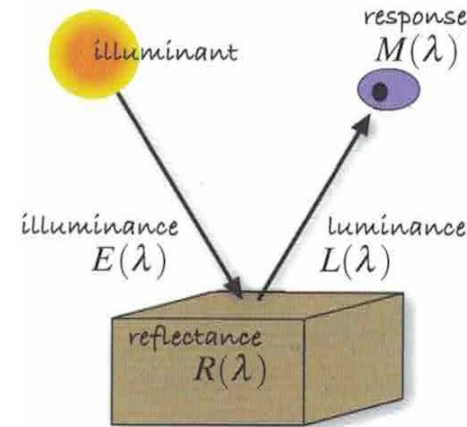
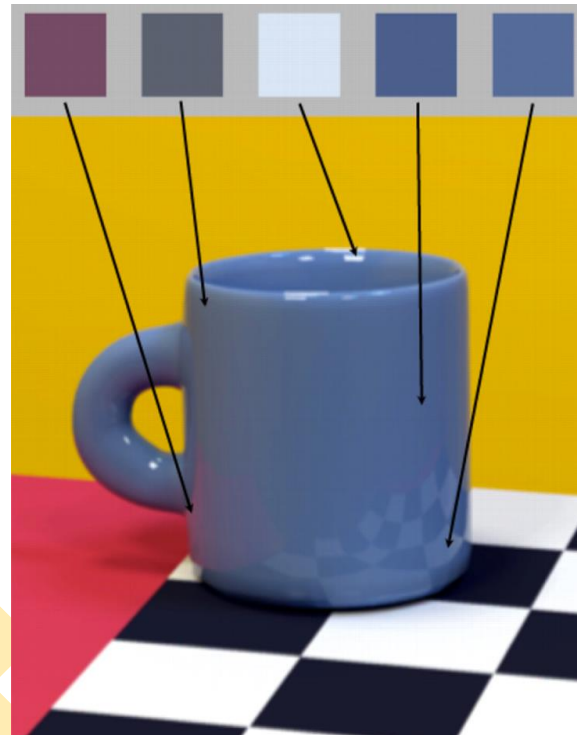
		Column															
		150					155						160				165
Row	50	183	183	181	184	177	200	200	189	159	135	94	105	160	174	191	196
		186	195	190	195	191	205	216	206	174	153	112	80	134	157	174	196
		194	196	198	201	206	209	215	216	199	175	140	77	106	142	170	186
		184	212	200	204	201	202	214	214	214	205	173	102	84	120	134	159
		202	215	203	179	165	165	199	207	202	208	197	129	73	112	131	146
		203	208	166	159	160	168	166	157	174	211	204	158	69	79	127	143
55		174	149	143	151	156	148	146	123	118	203	208	162	81	58	101	125
		143	137	147	153	150	140	121	133	157	184	203	164	94	56	66	80
		164	165	159	179	188	159	126	134	150	199	174	119	100	41	41	58
		173	187	193	181	167	151	162	182	192	175	129	60	88	47	37	50
		172	184	179	153	158	172	163	207	205	188	127	63	56	43	42	55
		156	191	196	159	167	195	178	203	214	201	143	101	69	38	44	52
60		154	163	175	165	207	211	197	201	201	199	138	79	76	67	51	53
		144	150	143	162	215	212	211	209	197	198	133	71	69	77	63	53
		140	151	150	185	215	214	210	210	211	209	135	80	45	69	66	60
		135	143	151	179	213	216	214	191	201	205	138	61	59	61	77	63
		135	143	151	179	213	216	214	191	201	205	138	61	59	61	77	63
	65	135	143	151	179	213	216	214	191	201	205	138	61	59	61	77	63

<https://www.dspguide.com/ch23/1.htm>

Appearance: Light and Colors

Perceived appearance is the result of

- Geometry
- Illumination
- Material properties



R
(G=0,B=0)

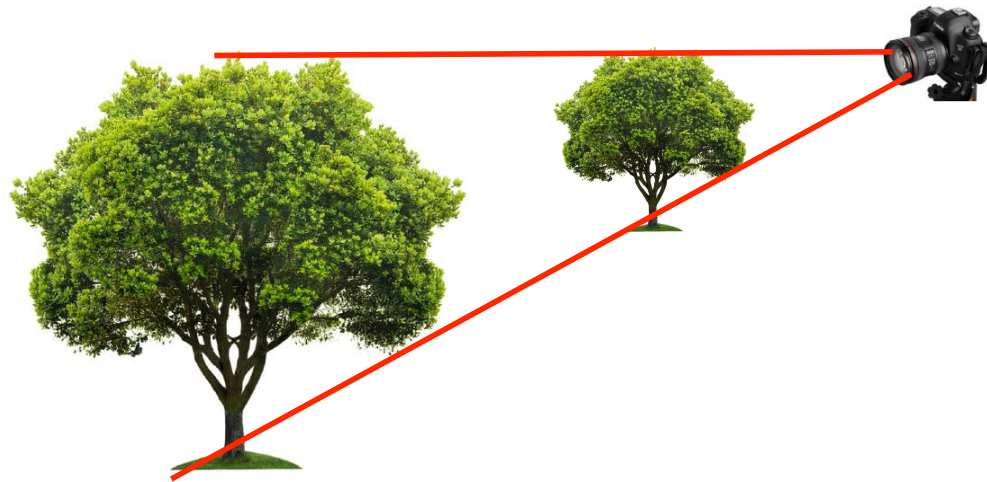


G
(R=0,B=0)



B
(R=0,G=0)

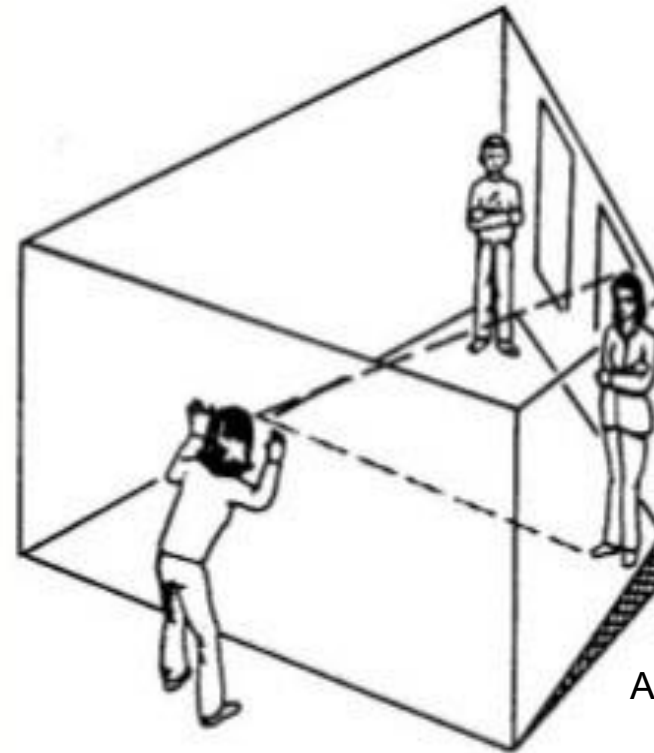
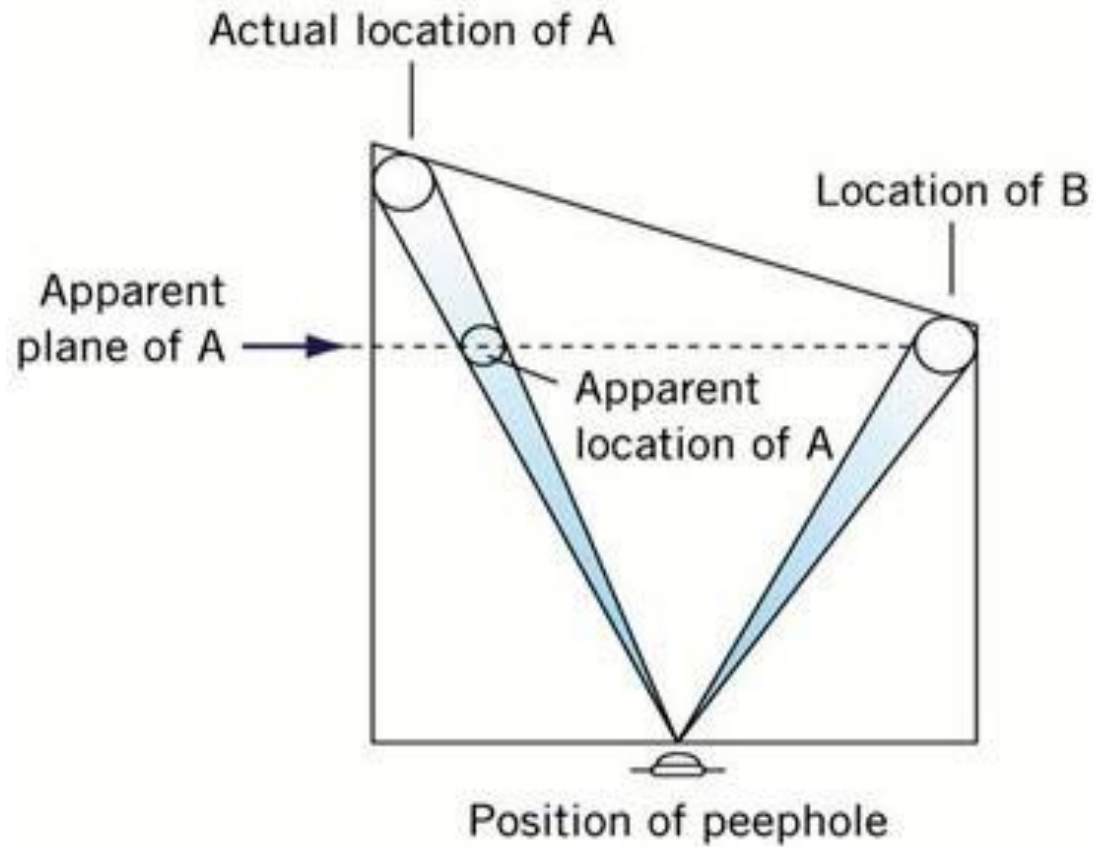
Image perspective projection: depth?



Ames Room (Illusion)



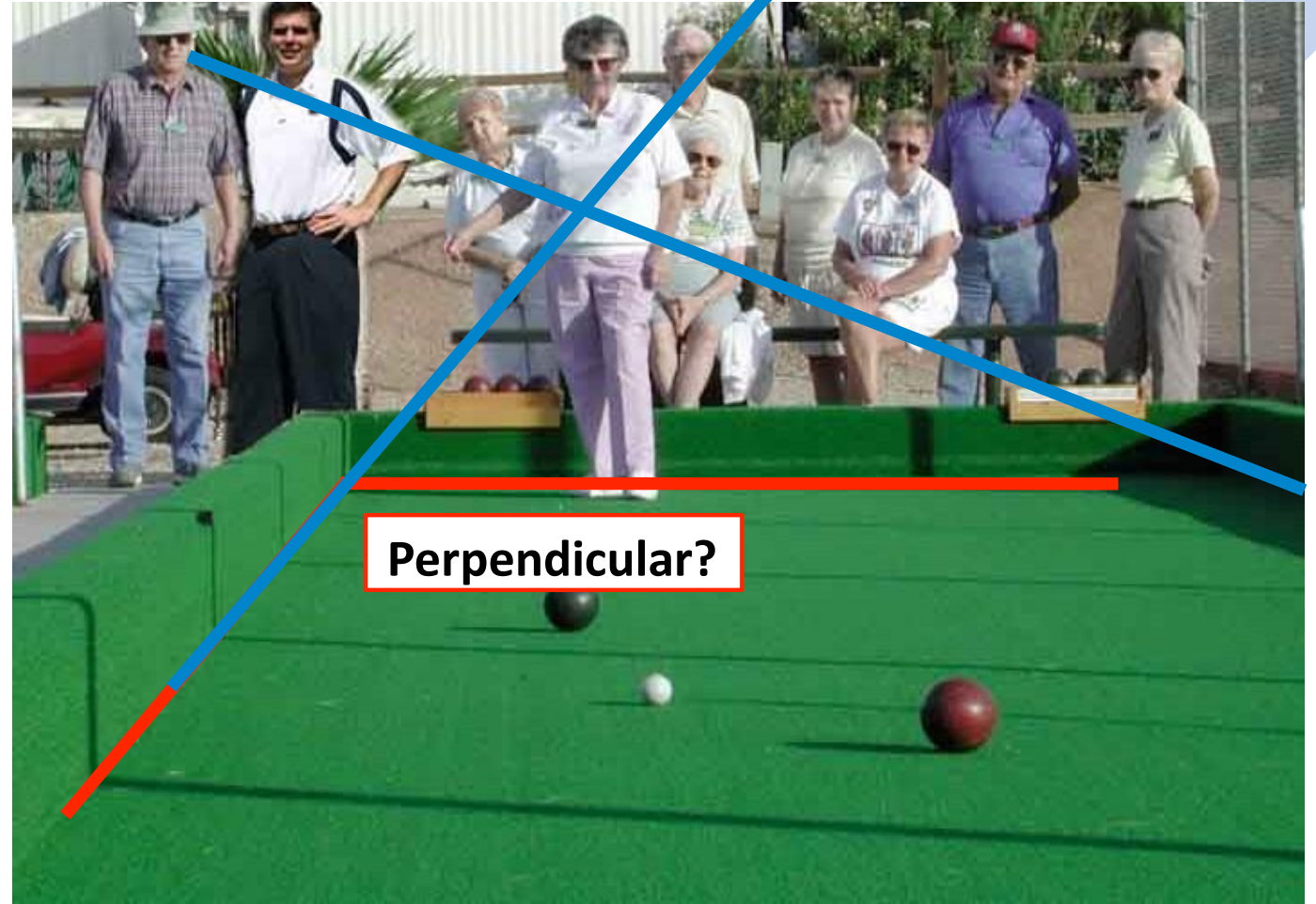
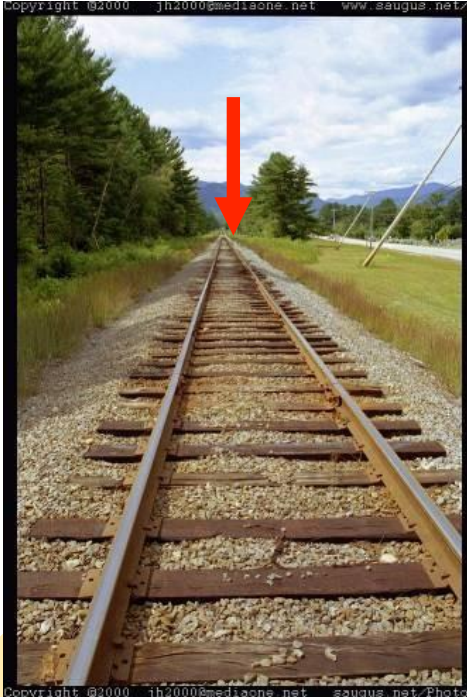
Ames Illusion



Ames, 1946

Image perspective projection: what is persevered, what not persevered?

- depth? length? Angles?
- straight lines remain straight



© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Parallel lines which intersect ...

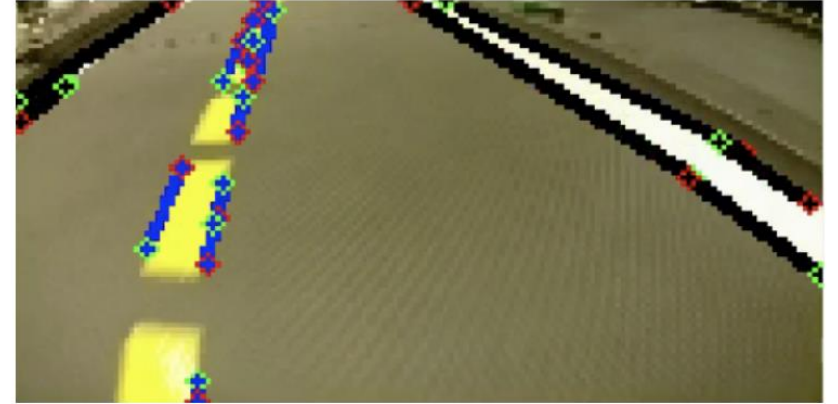
Slide adapted from
Frank Dellaert

Computer Vision:

Feature detection and tracking

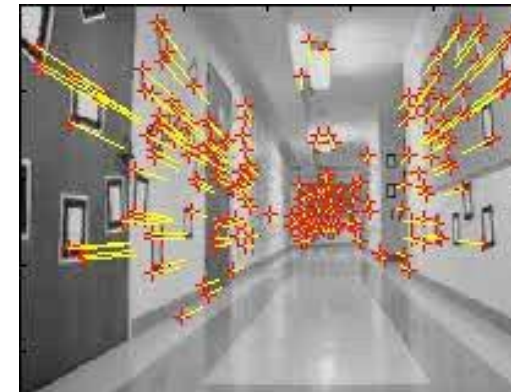
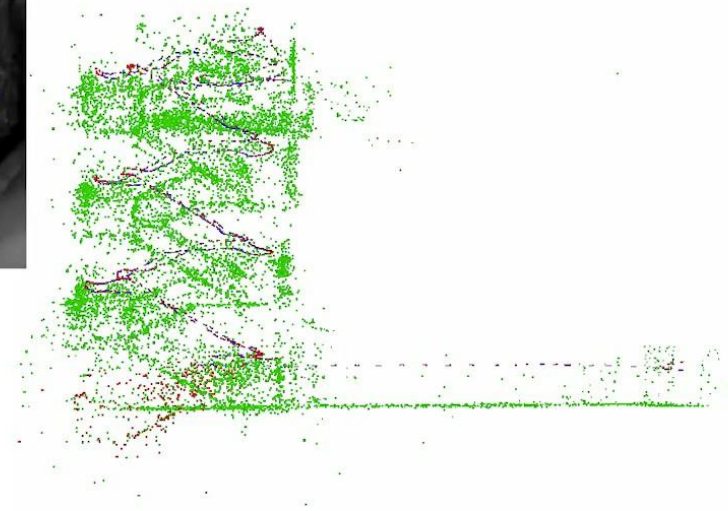
Feature detection

- **Definition:** a recognizable structure or pattern in the environment
 - lines, corners
 - geometric primitives (e.g., circles, squares)
 - objects (high-level features)
- **Why feature detection?**
- **Features are representing image's visual information**
- **Easier to describe mathematically: points, lines, ...**
- **Features help understand the image details for:**
 - **Matching images:** stereo vision, 3D construction and mapping
 - **Data compression:**
 - Image size (1000 by 1000 pixels) vs a segment is represented by 4 parameters
 - **Object detection for collision avoidance or object tracking**
 - **Classification**



Feature Detection -Motivation

- **Application**
 - Motion tracking
 - 3D reconstruction
 - Object recognition
 - ...



Corner Detection as a useful feature

- **corners:** also known as interest points, keypoints, or point features
 - easily identifiable points in the image
 - or: if given a corner in image I_1 , we can easily find corresponding pixel in I_2 (both images are picturing the same scene from different viewpoints)

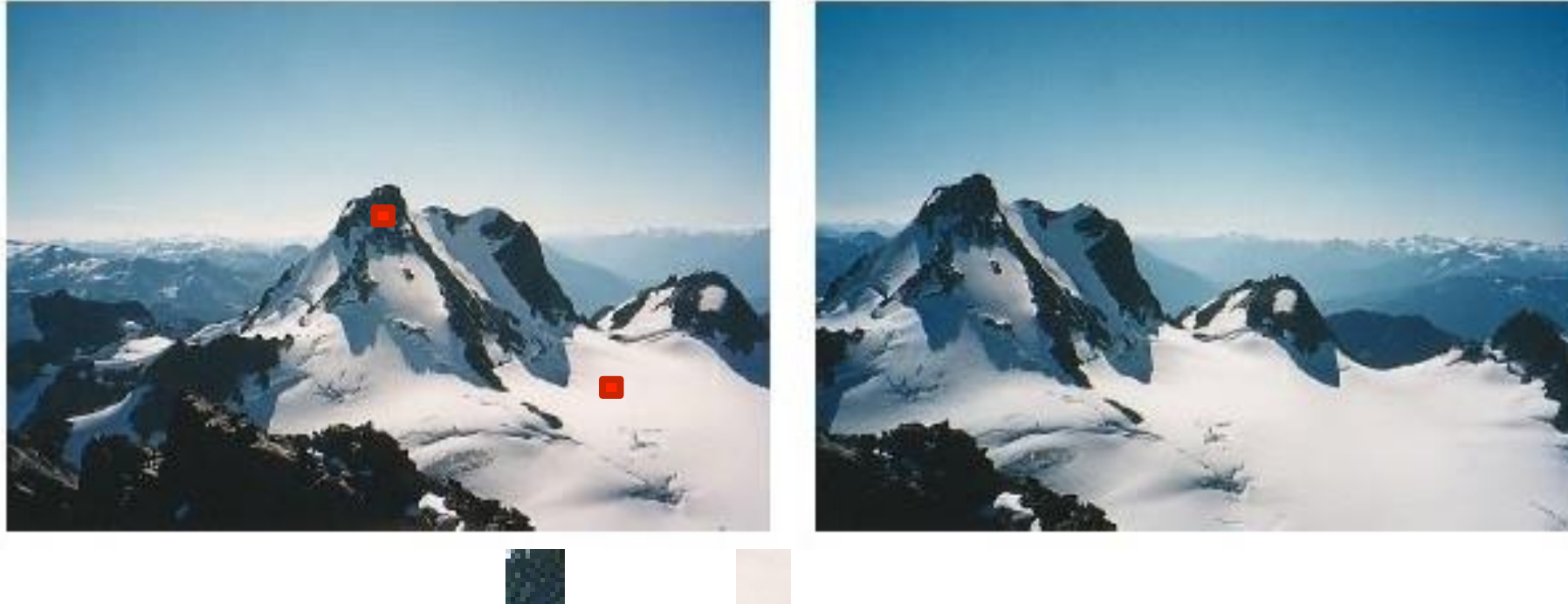
© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Corner Detection

- **corners:** also known as interest points, keypoints, or point features
 - easily identifiable points in the image
 - or: if given a corner in image I_1 , we can easily find corresponding pixel in I_2 (both images are picturing the same scene from different viewpoints)

© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Corner Detection

- **corners:** also known as interest points, keypoints, or point features
 - easily identifiable points in the image
 - or: if given a corner in image I_1 , we can easily find corresponding pixel in I_2 (both images are picturing the same scene from different viewpoints)

© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Corner Detection

- **corners:** also known as interest points, keypoints, or point features
 - easily identifiable points in the image
 - or: if given a corner in image I_1 , we can easily find corresponding pixel in I_2 (both images are picturing the same scene from different viewpoints)

© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Corner Detection

- **corners:** also known as interest points, keypoints, or point features
 - easily identifiable points in the image
 - or: if given a corner in image I_1 , we can easily find corresponding pixel in I_2 (both images are picturing the same scene from different viewpoints)

© source unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Image gradient

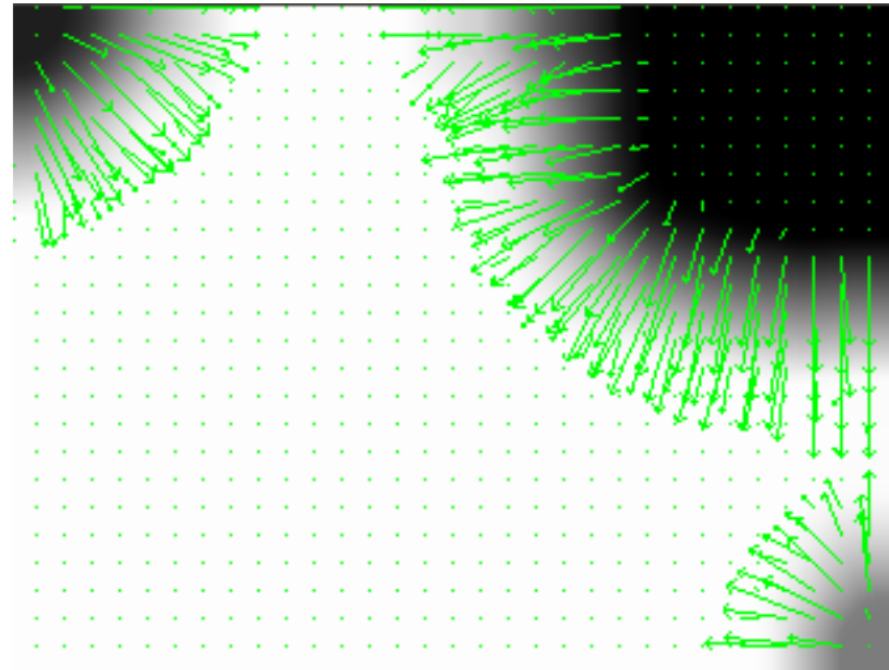
- Each pixel includes the intensity

$$\nabla \mathcal{I}(\mathbf{x}) = \nabla \mathcal{I}(u, v) = \begin{bmatrix} \frac{\partial \mathcal{I}(u, v)}{\partial u} \\ \frac{\partial \mathcal{I}(u, v)}{\partial v} \end{bmatrix}$$



From gradients to finite differences (pract

$$\nabla \mathcal{I}(\mathbf{x}) = \nabla \mathcal{I}(u, v) \approx \begin{bmatrix} \frac{\mathcal{I}(u+h, v) - \mathcal{I}(u, v)}{h} \\ \frac{\mathcal{I}(u, v+h) - \mathcal{I}(u, v)}{h} \end{bmatrix}$$



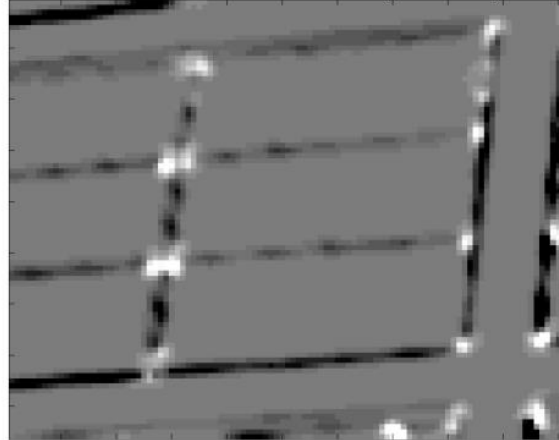
Corner Detection

- we can compute a “corneriness score” at each pixel in the image
- peaks are the most distinguishable corners

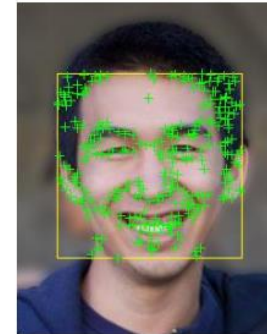
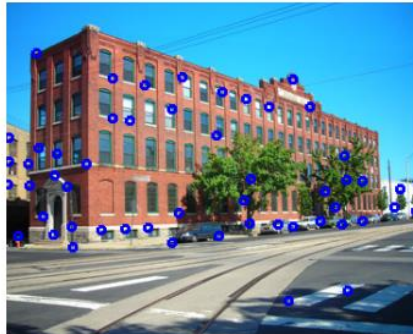
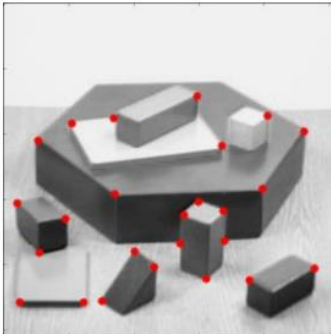
original image



corneriness score (Harris)



peaks

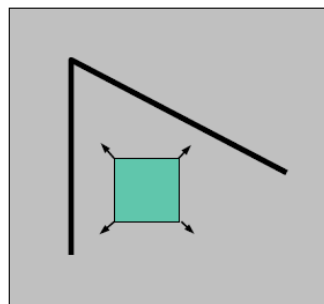
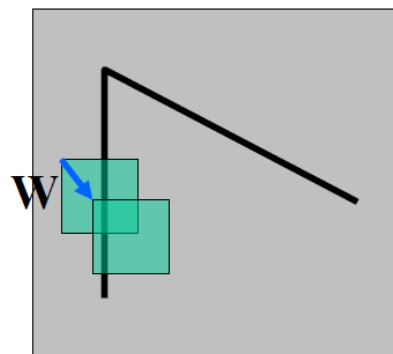


Corner Detection

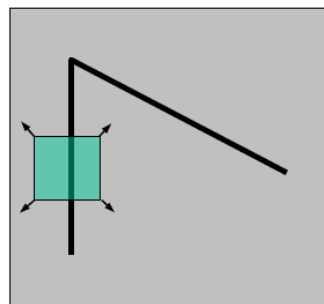
- finding corners in images:

- Consider shifting window $\mathbf{W}$ by δ
 - How do the pixels in $\mathbf{W}$ change?
 - compare the windows using **sum of squared differences** (SSD) error:

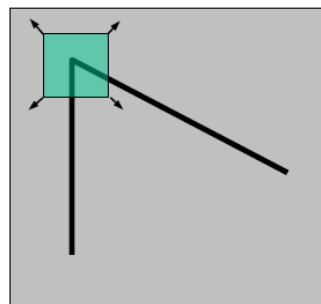
$$\sum_{\mathbf{x} \in W(\bar{\mathbf{x}})} \|\mathcal{I}(\mathbf{x} + \delta) - \mathcal{I}(\mathbf{x})\|^2$$



“flat” region:
no change in all
directions



“edge”:
no change along the
edge direction



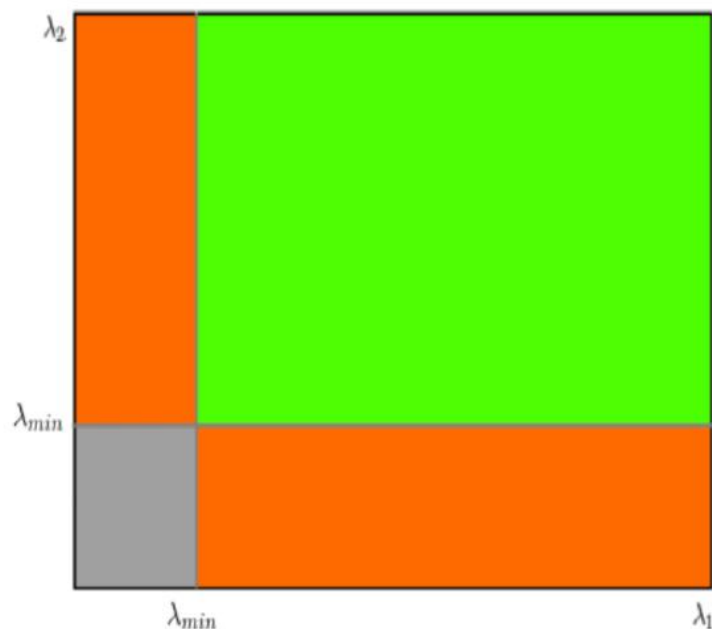
“corner”:
significant change in all
directions, i.e., even the
minimum change is large

$$\bar{\mathbf{x}} = \begin{bmatrix} u \\ v \end{bmatrix} \rightarrow \mathbf{G} = \sum_{\mathbf{x} \in W(\bar{\mathbf{x}})} \nabla \mathcal{I}(\mathbf{x}) \nabla \mathcal{I}(\mathbf{x})^\top$$

Cornerness Scores

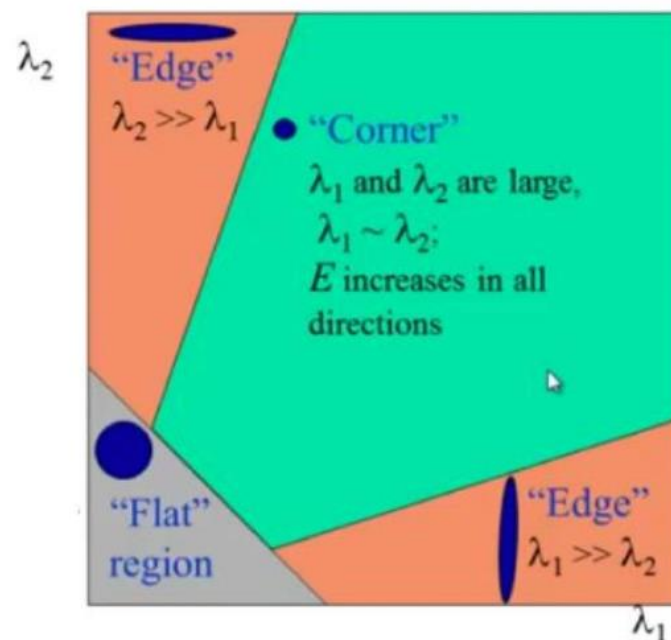
$$\bar{\mathbf{x}} = \begin{bmatrix} u \\ v \end{bmatrix} \Rightarrow \mathbf{G} = \sum_{\mathbf{x} \in W(\bar{\mathbf{x}})} \nabla \mathcal{I}(\mathbf{x}) \nabla \mathcal{I}(\mathbf{x})^\top$$

Calling λ_1 and λ_2 the eigenvalues of the matrix $\mathbf{G}$



$$S(\mathbf{G}) = \lambda_{\min}(\mathbf{G})$$

Shi-Tomasi corner
detector



$$C(\mathbf{G}) = \det(\mathbf{G}) - k \operatorname{tr}(\mathbf{G})^2$$

Harris corner
detector

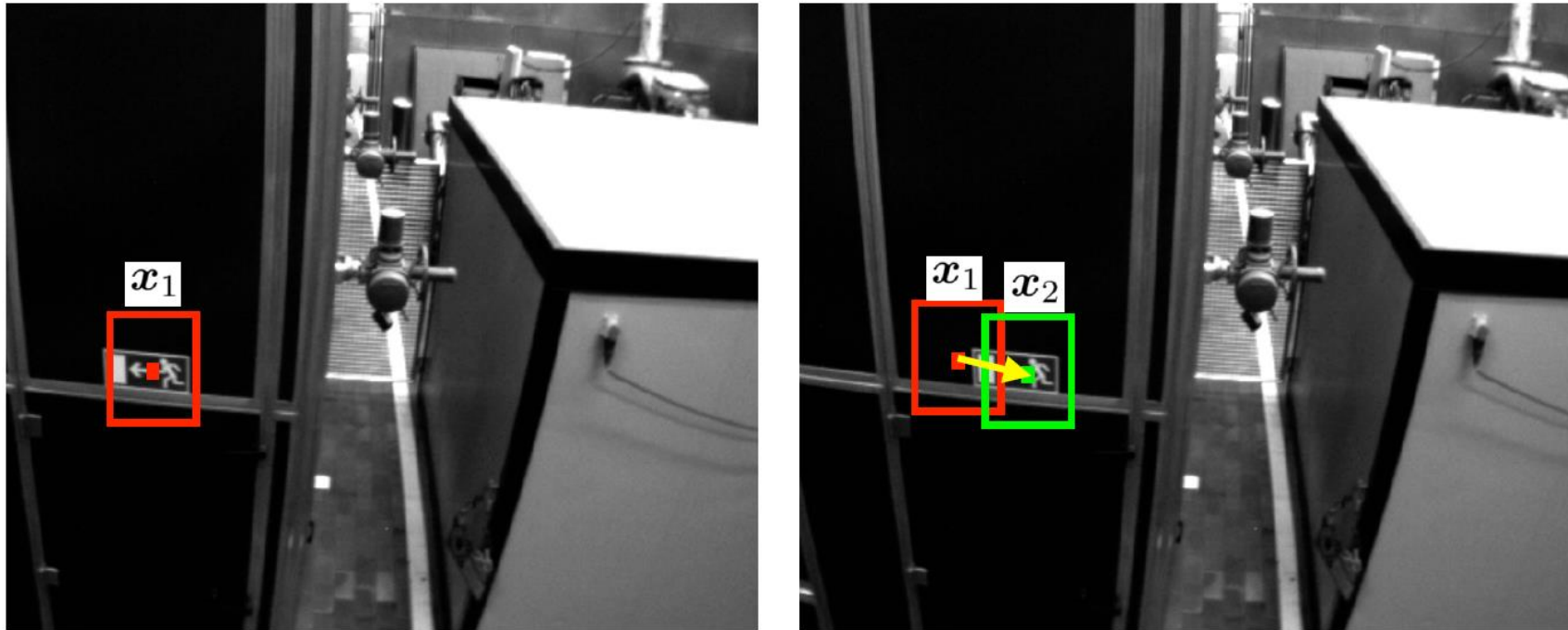
Correspondence

given a corner in image I_1 (and its neighborhood),
how can we find corresponding pixel in I_2 ?



- **Feature tracking** (~ optical flow)
- **Feature matching** (descriptor-based)

Feature Tracking

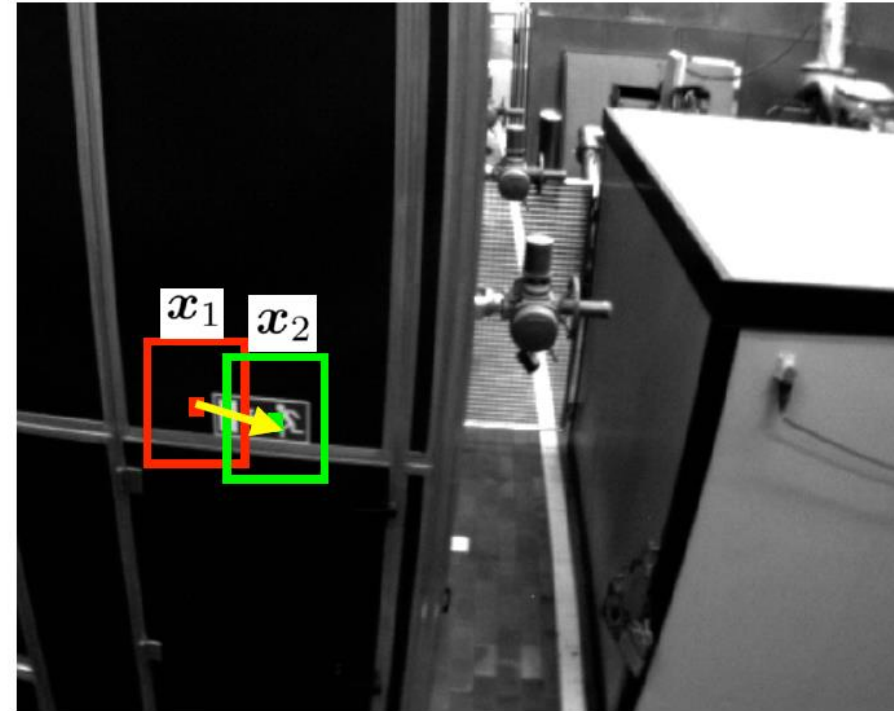


Computing the corresponding pixel (x_2) is the same as computing the displacement δ

$$x_2 = x_1 + \delta$$

(translational motion model)

Feature Tracking



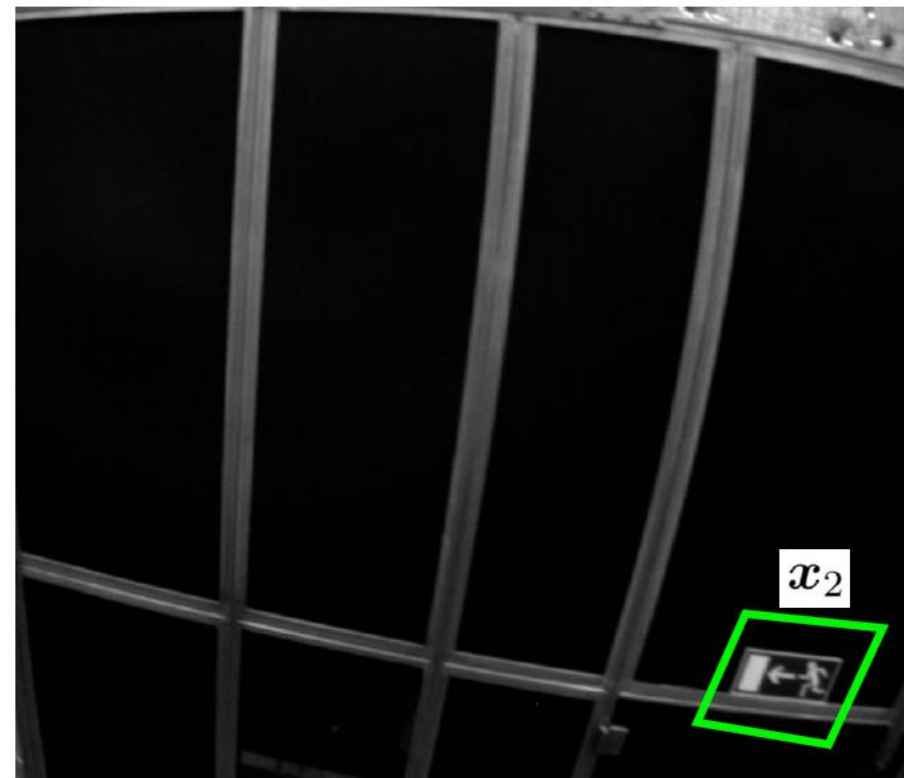
Computing the corresponding pixel (x_2) is the same as computing the displacement δ

Idea: find the displacement δ such that the intensity of points in the neighborhood of y in I_1 is similar to the one in I_2

$$\min_{\delta} \sum_{y \in W(x_1)} \|\mathcal{I}_1(y) - \mathcal{I}_2(y + \delta)\|^2$$

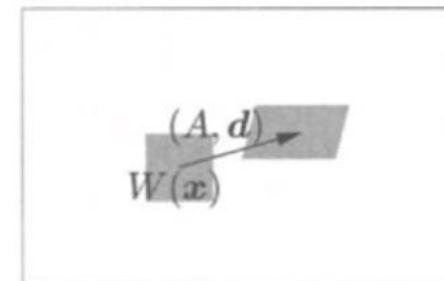
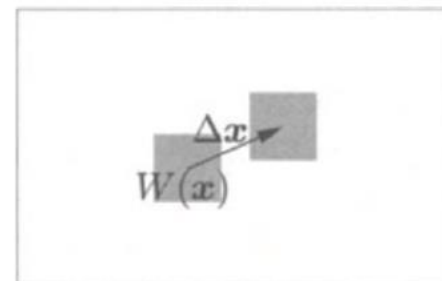
(translational motion model)

Feature Tracking

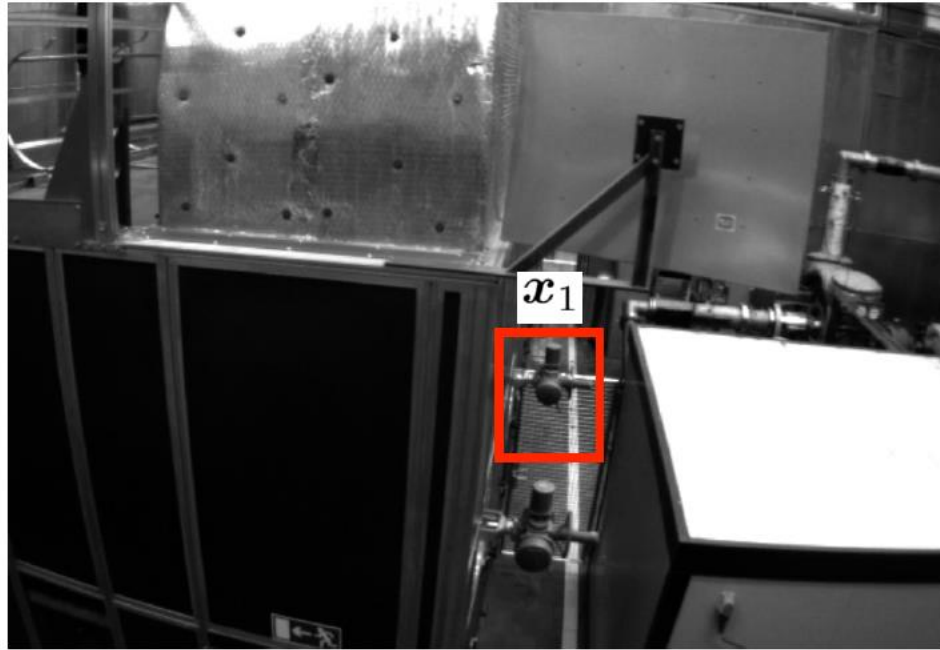


$$\min_{A, \delta} \sum_{y \in W(x_1)} \|\mathcal{I}_1(y) - \mathcal{I}_2(Ay + \delta)\|^2$$

(affine motion model)

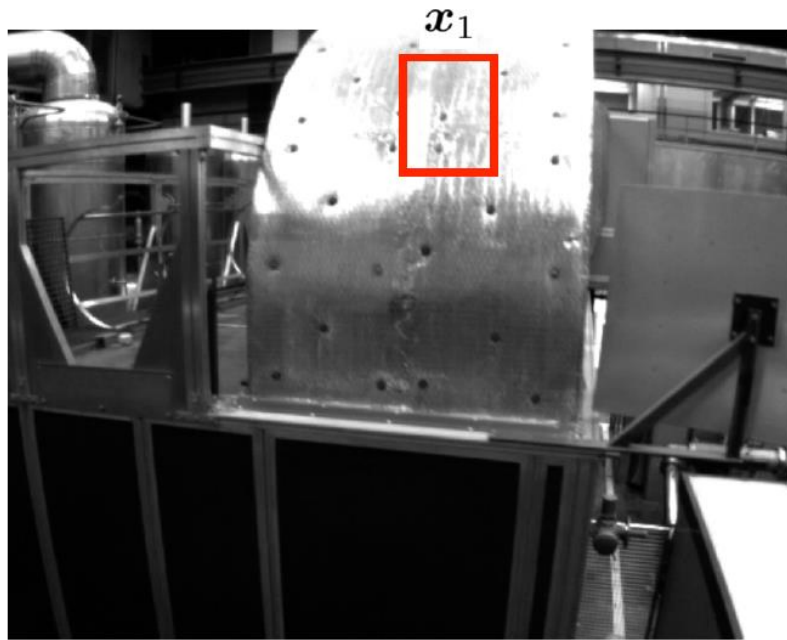


Hidden Assumptions...



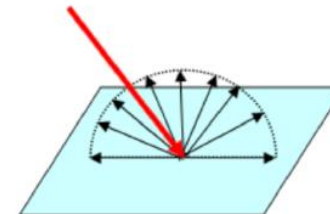
Pixel motion models not valid in presence of occlusions

Hidden Assumptions...



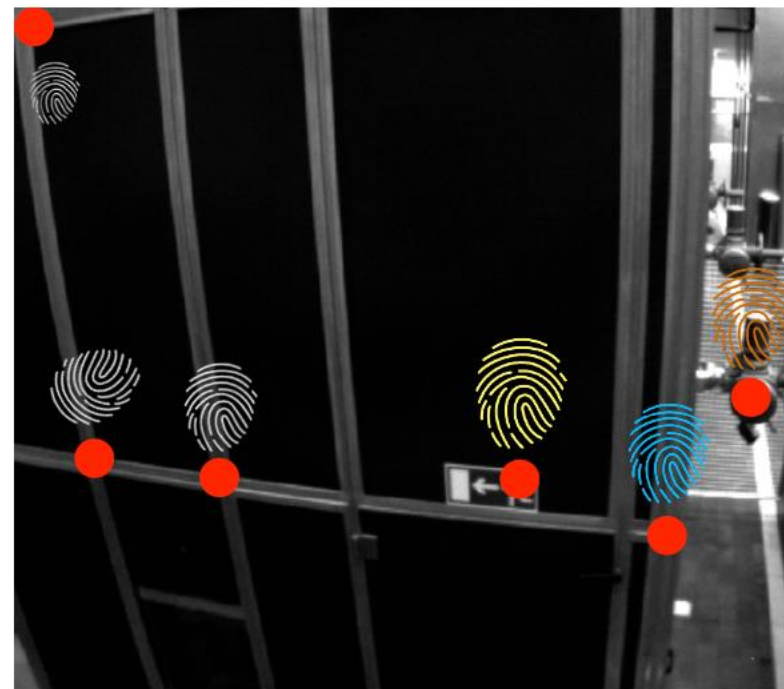
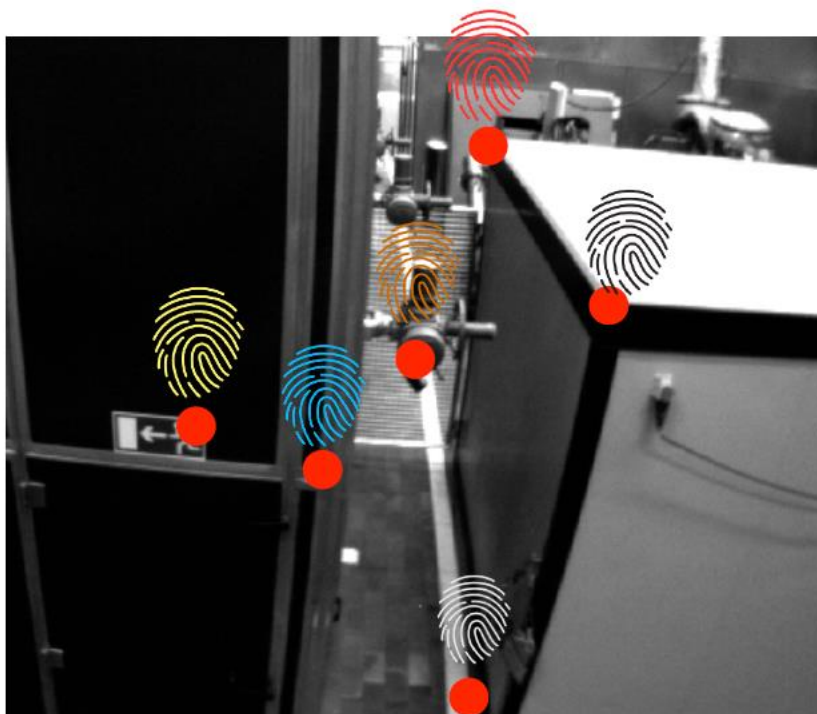
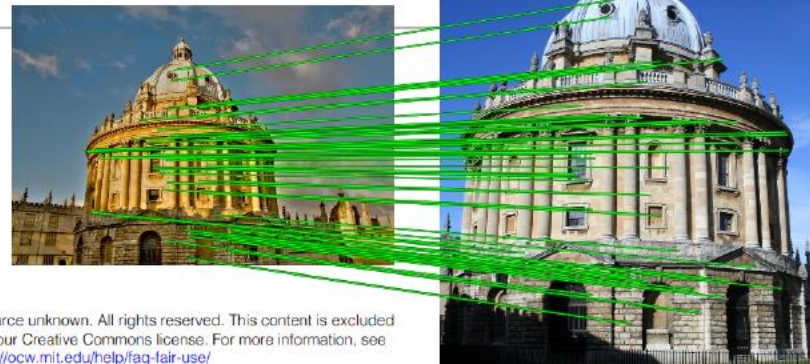
Matching image patches assume that the brightness
does not change due to viewpoint changes
(**brightness constancy constraints**)

True for Lambertian surfaces



Descriptor-based Feature Matching

Feature tracking does not typically work for large changes of viewpoint
(**large baseline**)

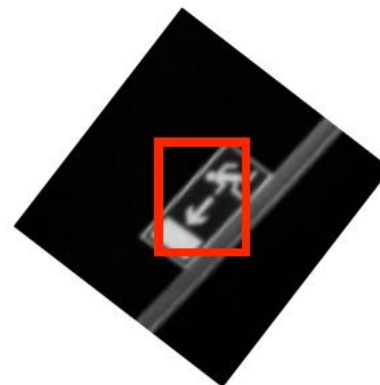
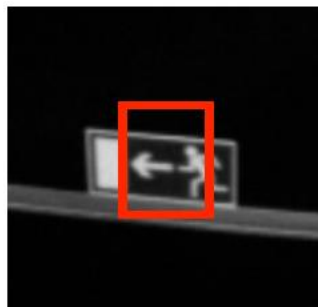


Descriptor is a signature we attach to a (point) feature,

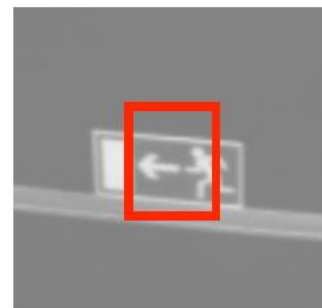
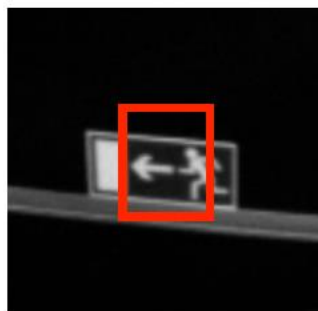
Ideal Properties of a Detector/Descriptor

Rotation invariance

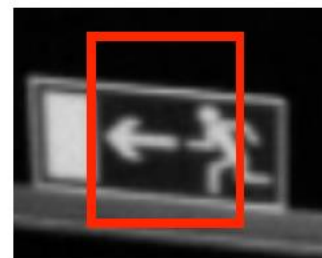
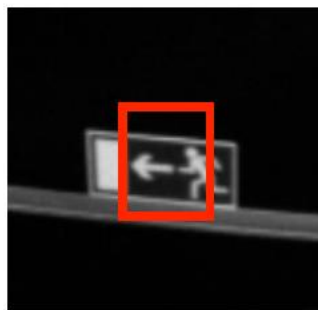
(more generally:
Viewpoint invariance)



Illumination invariance

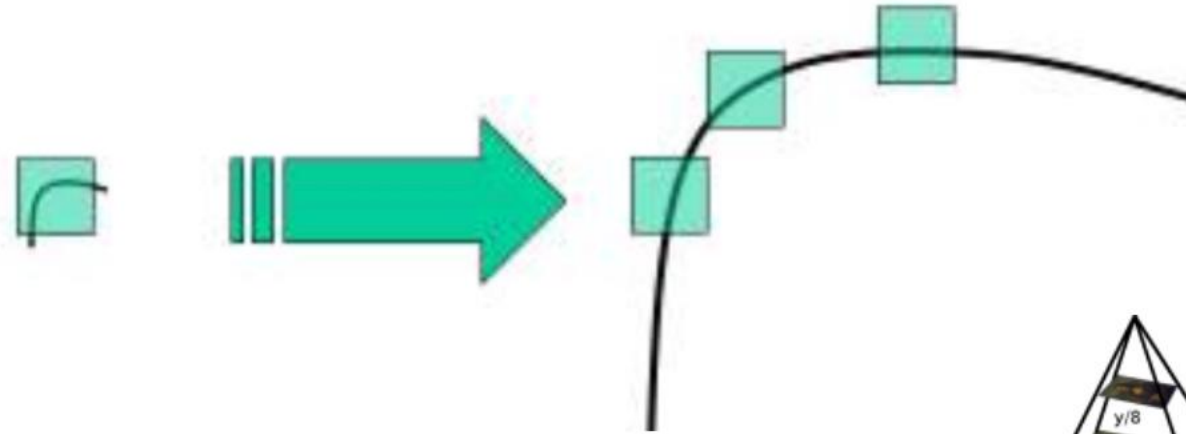


Scale invariance

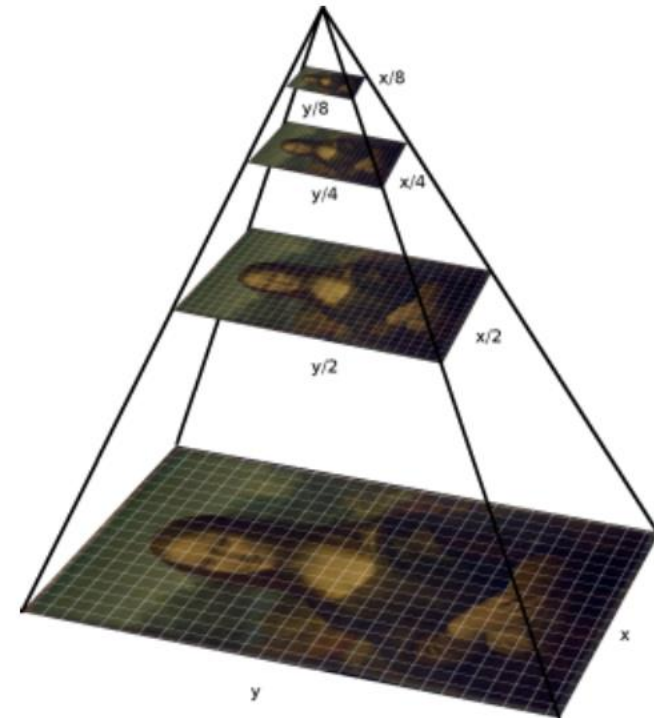


(more in the Lab 5 handouts: repeatability, efficiency ..)

Are Harris Corners **Scale** invariant?



- **Other detectors have been proposed:**
huge literature:
SIFT, SURF, ORB, BRIEF, MSER, ...
- **blob detectors:**
process the image at different scales



Nice tutorial on SIFT:

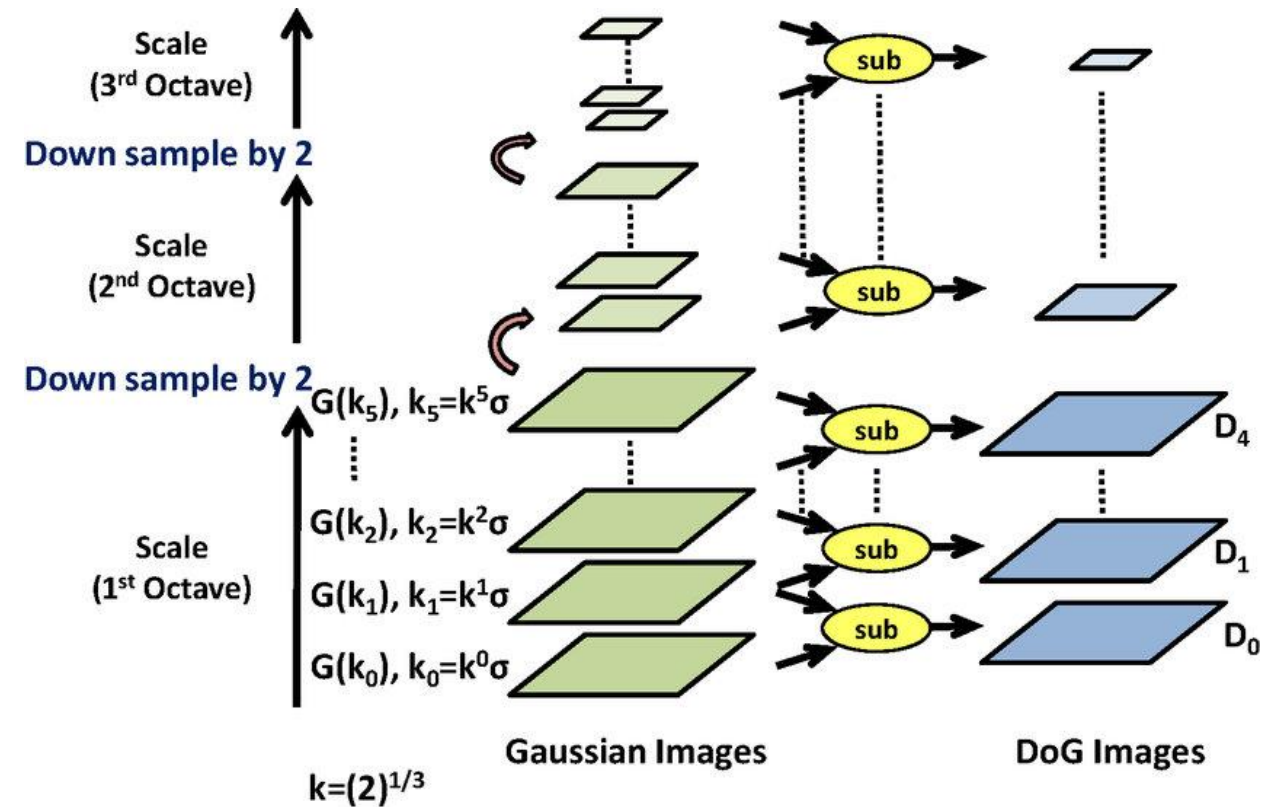
SIFT – Scale Invariant Feature Transform

- Goal: Find a scale and orientation invariant feature
- SIFT Steps:
 - Find the interest points using LoG (Laplacian of Gradient) which is approximated by DoG (Difference of Gaussian) → called scale space peak selection
 - Reject the outliers and finalize keypoints/interest points using second-order Hessian matrix
 - Assign Orientation to each keypoint
 - Create SIFT descriptor (vector of 128 dimension) for each key point

First step: find keypoints

- **Gaussian Blurring:**

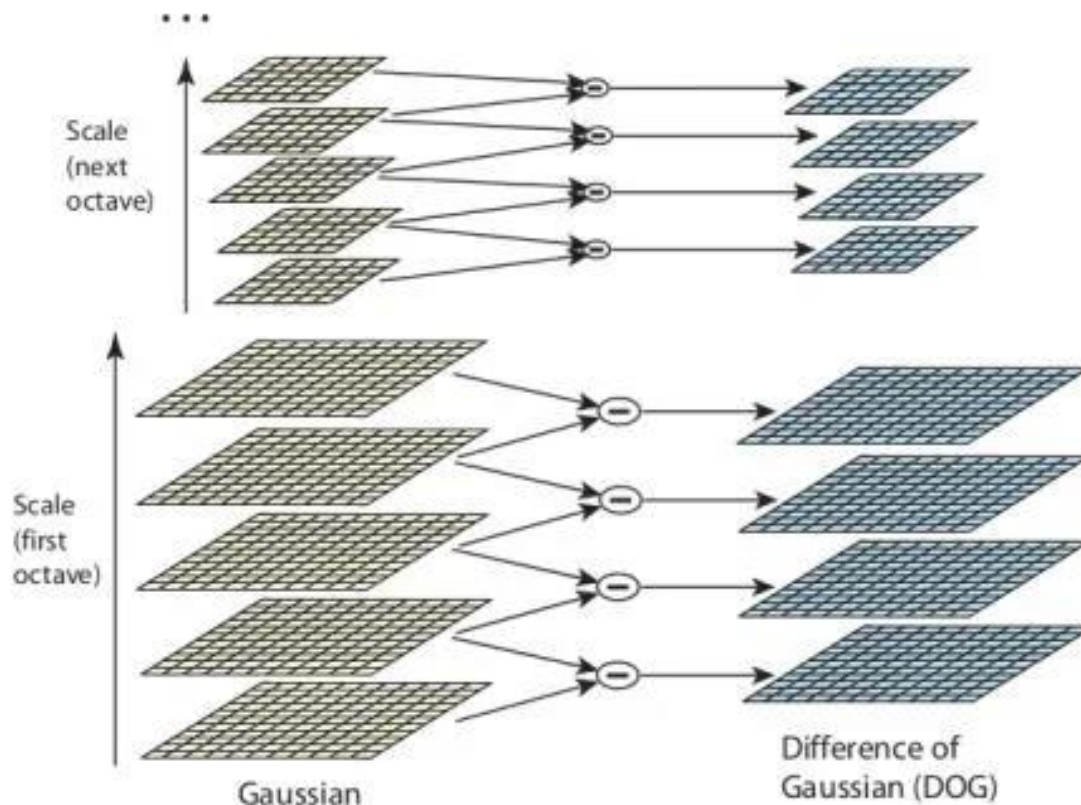
- In imaging science, **difference of Gaussians (DoG)** is a feature enhancement algorithm that involves the subtraction of one Gaussian blurred version of an original image from another, less blurred version of the original (source: Wikipedia)
- Within an octave, images are progressively blurred using the Gaussian Blur operator.



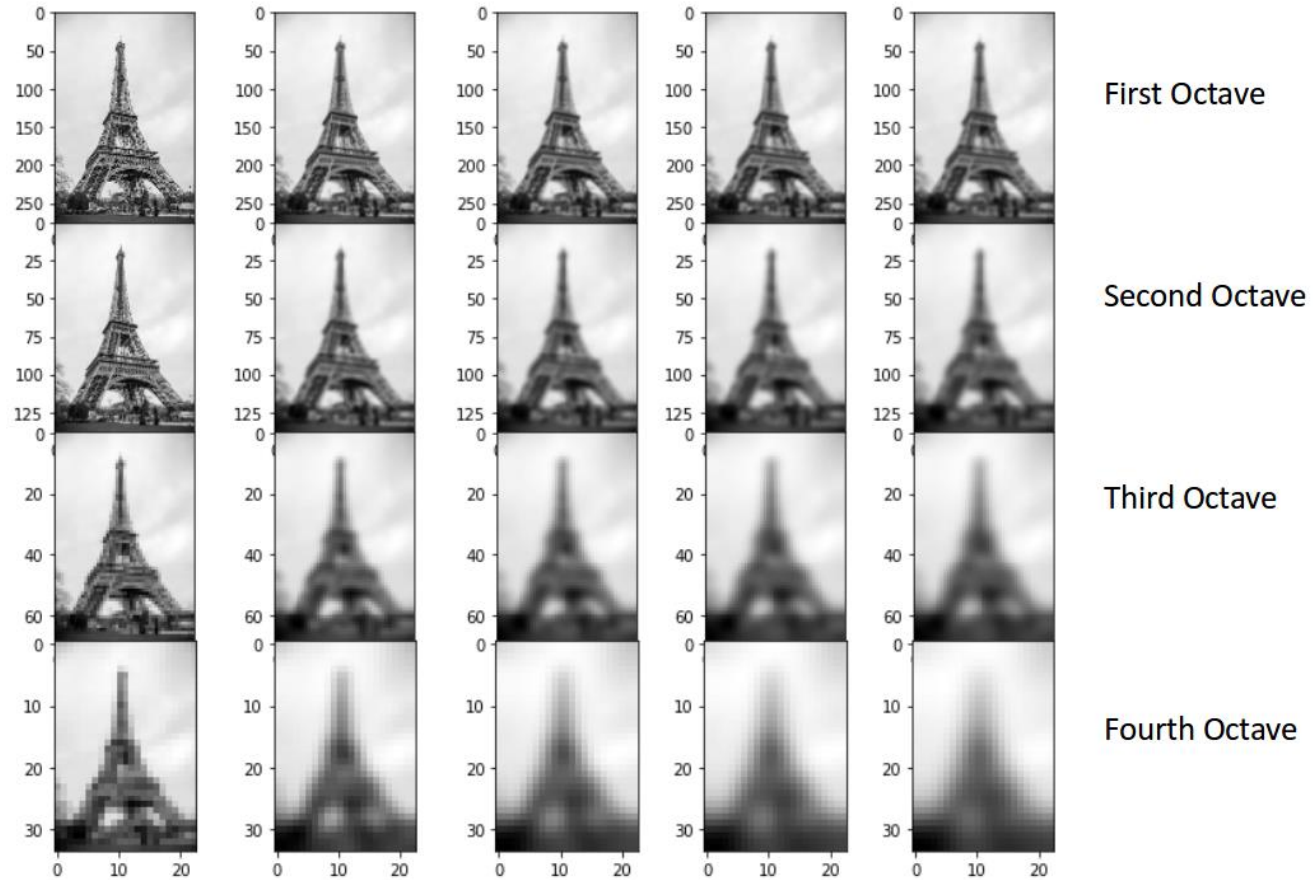
First step: find keypoints - continued

- **DOG(Difference of Gaussian kernel)**

The difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ



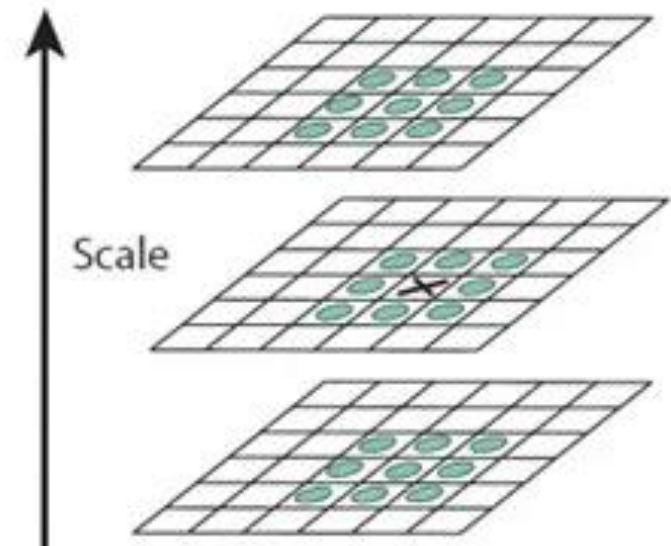
Example on DoG steps



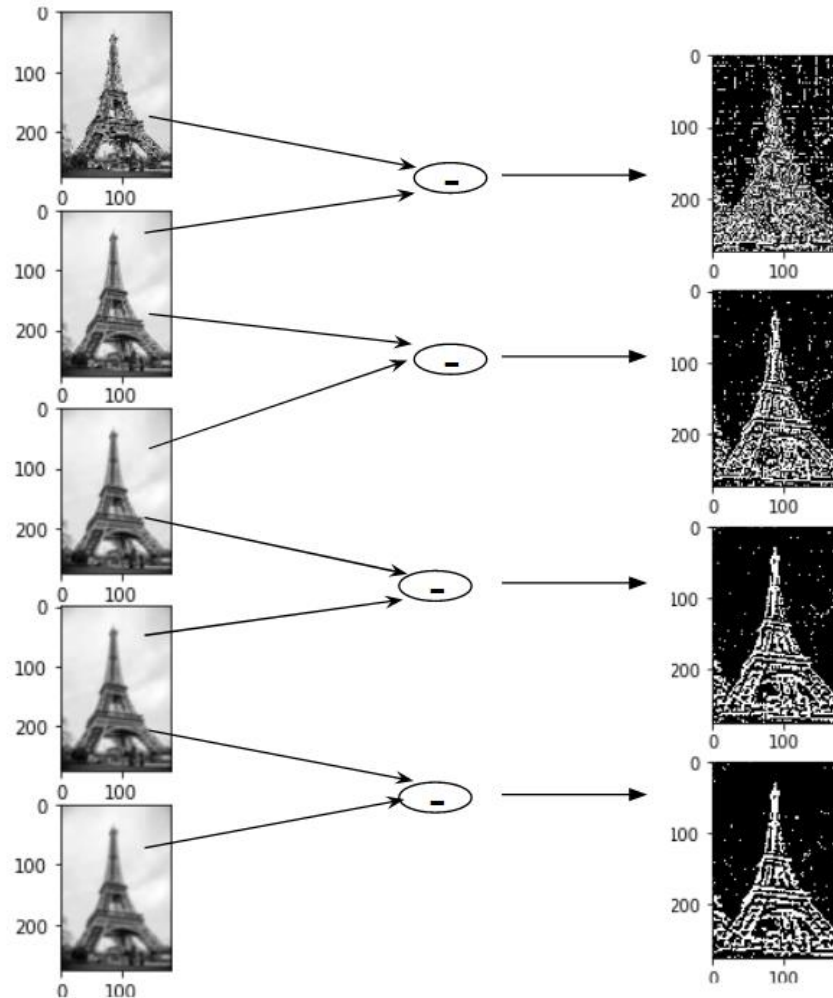
Finding keypoints

- One pixel in an image is compared with its 8 neighbors as well as 9 pixels in the next scale and 9 pixels in previous scales (total of 26 pixels)
- If it is a local extrema(has the maximum or minimum intensity), it is a potential keypoint
- Issue: some keypoints are outlier
- Next step: remove outliers using hessian

<https://www.analyticsvidhya.com/blog/2019/10/detailed-guide-powerful-sift-technique-image-matching-python/>



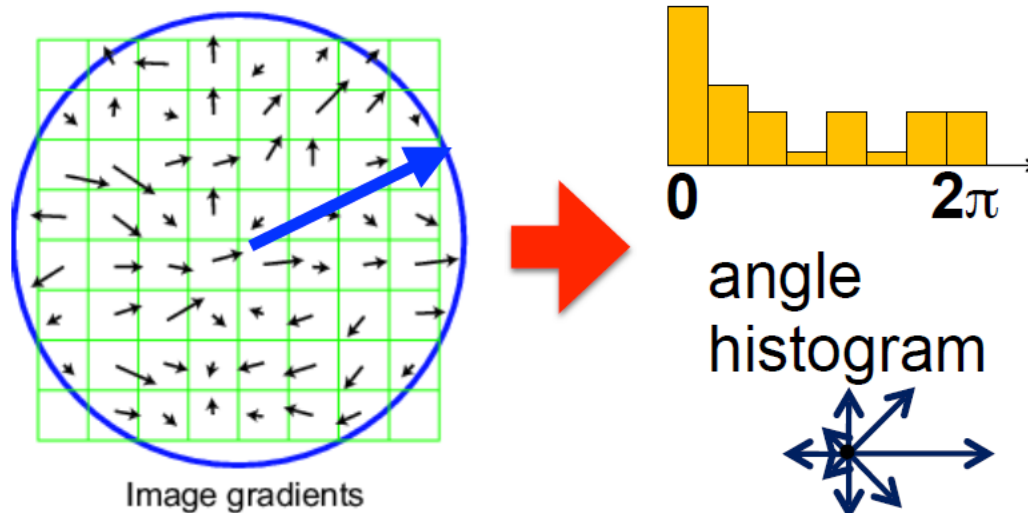
Example on DoG steps



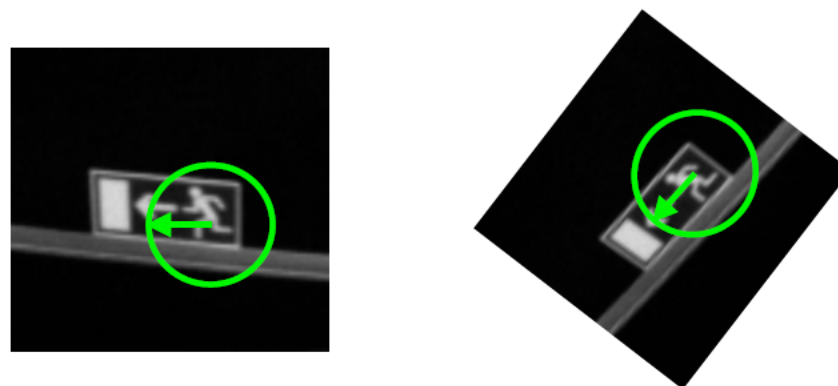
Example: SIFT Descriptor (1/2)

SIFT: Scale-Invariant Feature Transform

- Take 16x16 square window around detected feature
- Compute gradient orientation and magnitude for each pixel
- Create histogram of gradients weighted by magnitude
- Peak is **orientation** of feature



© Dr. Mubarak Shah. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



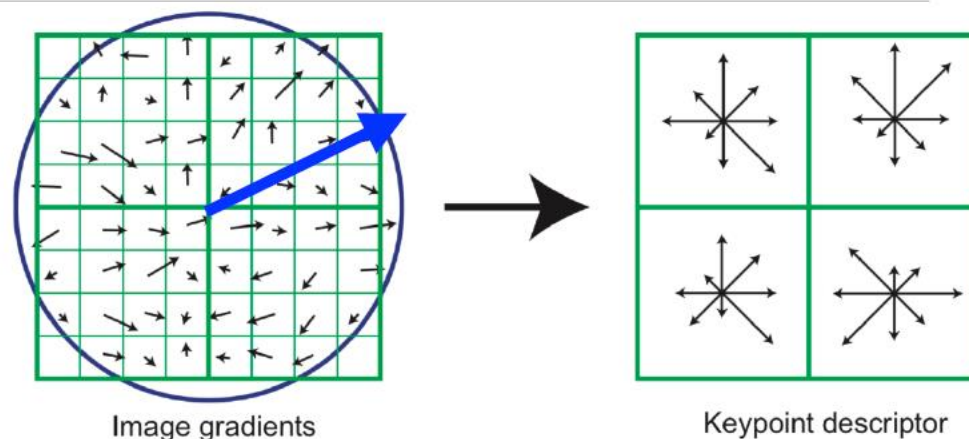
Lowe, David G. (1999). "Object recognition from local scale-invariant features", CVPR'99

D. G. Lowe, "Object recognition from local scale-invariant features," Proceedings of the Seventh IEEE International Conference on Computer Vision, Kerkira, Greece, 1999, pp. 1150-1157 vol.2, doi: 10.1109/ICCV.1999.790410 © IEEE. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Example: SIFT Descriptor (2/2)

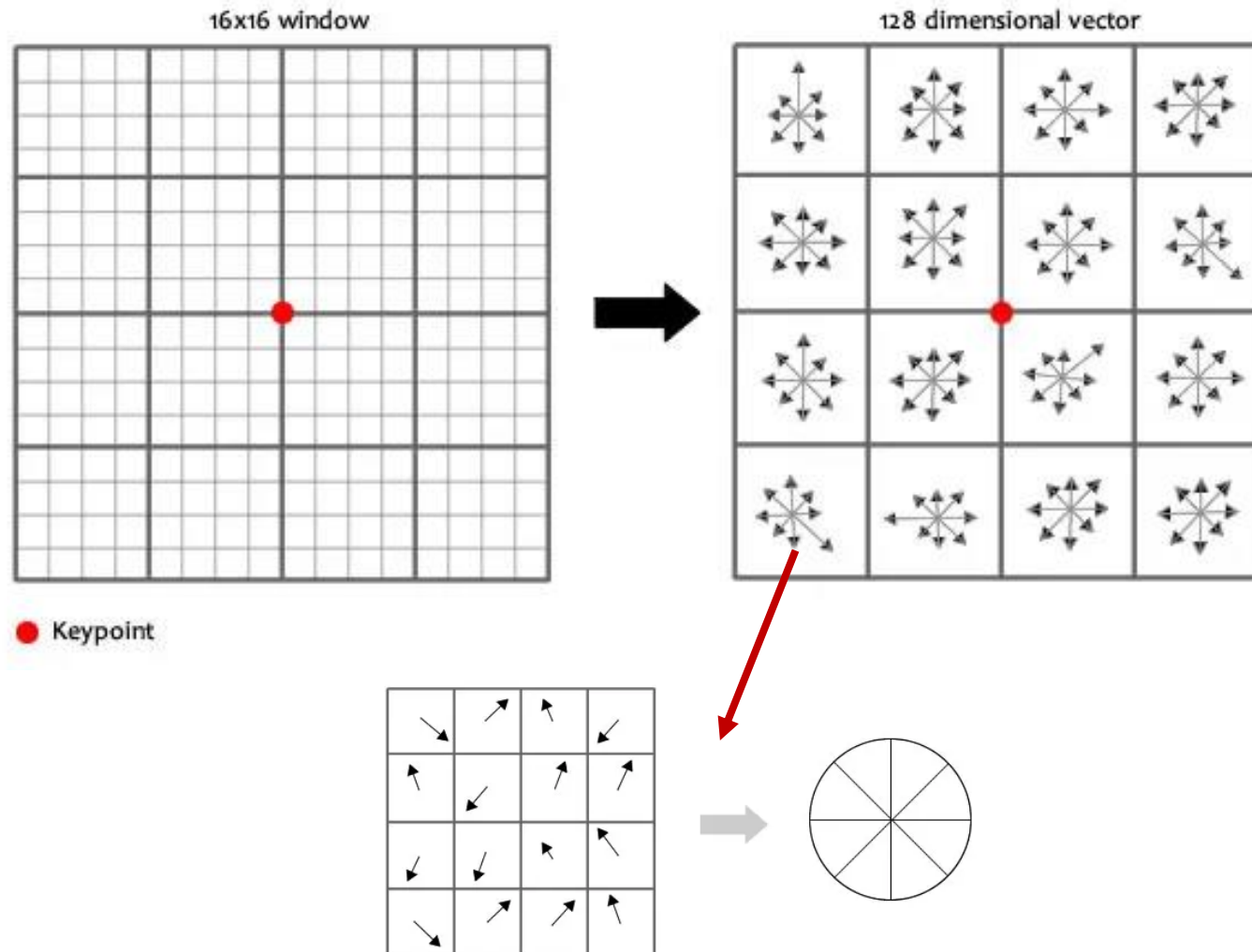
How to get SIFT descriptor?

- Transform all gradients with respect to (main) orientation
- Split window in 16 squares and for each compute a histogram with 8 sectors
- Stack histogram into a **descriptor** vector of $16 \times 8 = 128$ scalars
- Normalize to have norm = 1



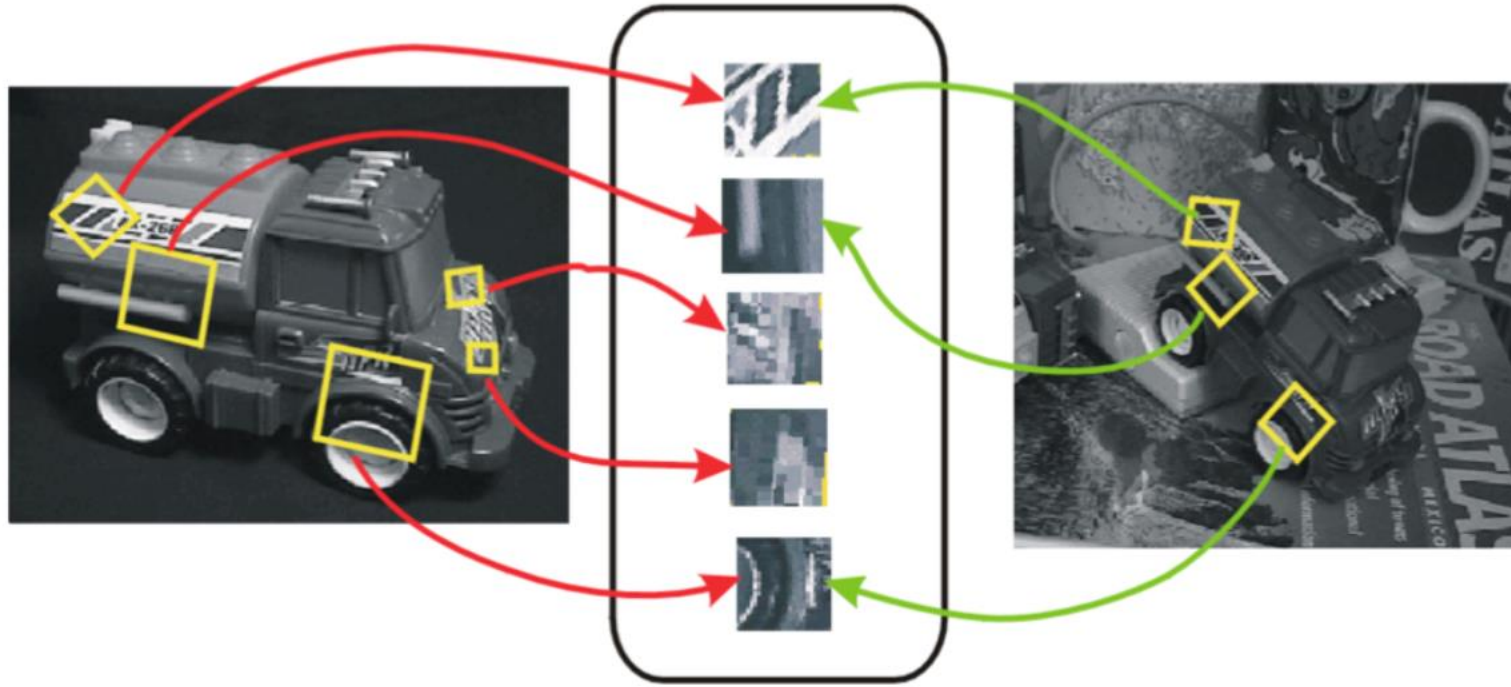
Lowe, David G. (1999). "Object recognition from local scale-invariant features", CVPR'99

Looking closer to SIFT dimension...



$$16 \times 8 = 128$$

Feature Matching



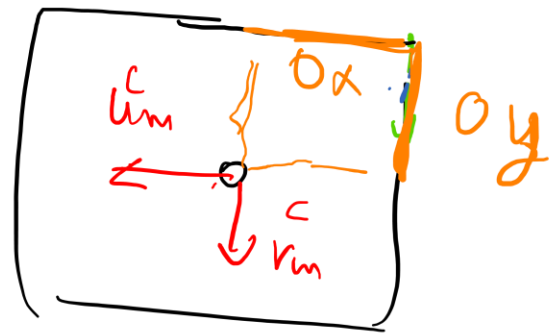
© Frank Dellaert. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

- For each descriptor in I_1 find closest descriptor in I_2 (nearest neighbor)
- Speed up with **approximate** nearest neighbor algorithms (FLANN library)

Some references on nearest neighbor algorithm

- [https://en.wikipedia.org/wiki/Nearest neighbour algorithm](https://en.wikipedia.org/wiki/Nearest_neighbour_algorithm)
- <https://scikit-learn.org/stable/modules/neighbors.html>

Backup notes

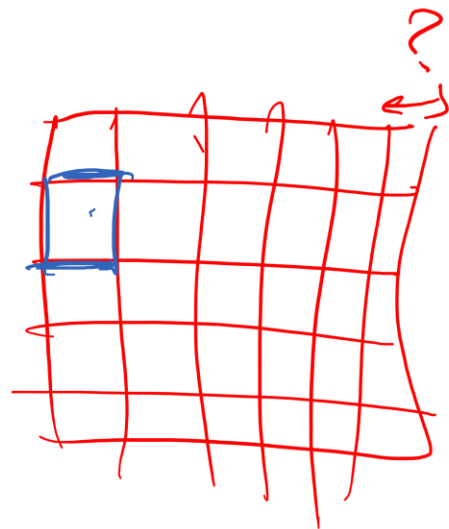


P_{2D}^C

$$\rightarrow (u_m^c, v_m^c)$$

projection

$\Rightarrow$ meter



Picked $\Leftrightarrow$ meter

$$S_u \neq S_y$$

$$u^I = S_u u_m^c + O_u$$

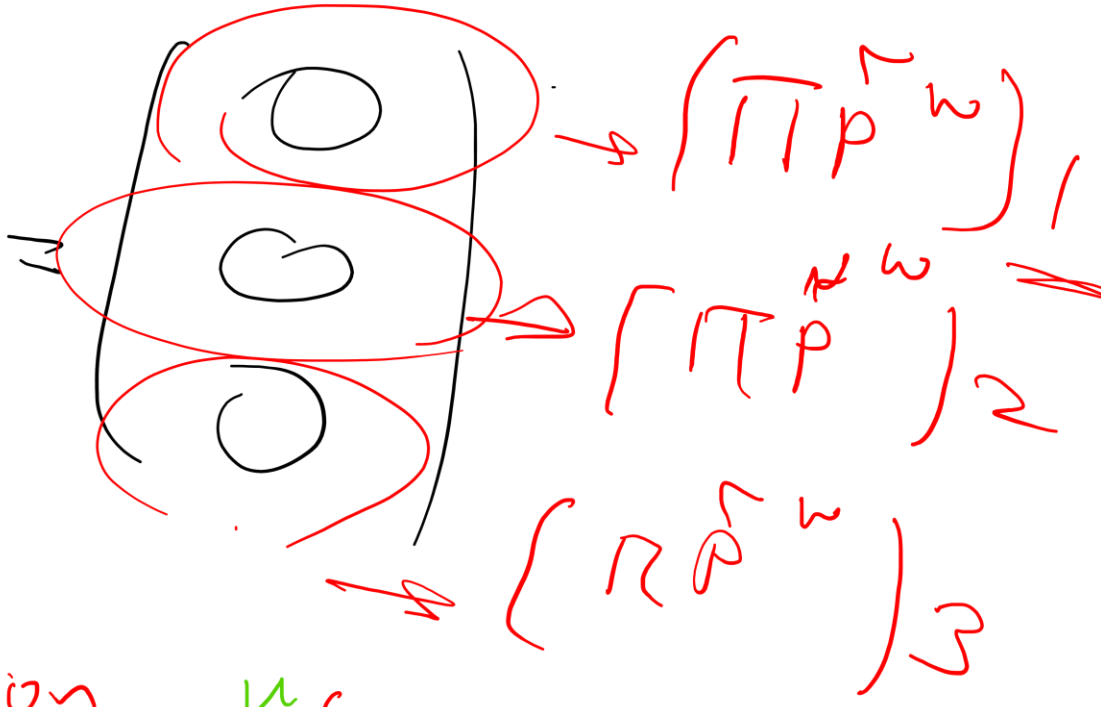
$$v^I = S_y v_m^c + O_y$$

$$\begin{pmatrix} u^I \\ v^I \\ 1 \end{pmatrix} = \begin{pmatrix} S_u & S_y & O_u \\ 0 & S_y & O_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u_m^c \\ v_m^c \\ 1 \end{pmatrix}$$

 square: $S_u = S_y$
 rectangle: $S_u \neq S_y$

 $\Rightarrow$ so skewed $\neq 0$

$$u^I = \frac{[\Pi \tilde{P}^w]_1}{[\Pi \tilde{P}^w]_3}, \quad v^I = \frac{[\Pi \tilde{P}^w]_2}{[\Pi \tilde{P}^w]_3}$$

$$\underline{P_z^c} \begin{pmatrix} u^I \\ v^I \\ 1 \end{pmatrix} = \underline{\Pi \tilde{P}^w}$$


$\rightarrow [\Pi \tilde{P}^w]_1$
 $\rightarrow [\Pi \tilde{P}^w]_2$
 $\rightarrow [\Pi \tilde{P}^w]_3$

Distortion: N.I. 4

$r \Rightarrow$ radial distortion

$$u^c = (1 + a_1 r^2 + a_2 r^4) u^I \text{ distort}$$

$$P_z^c \begin{pmatrix} u_m^c \\ v_m^c \\ 1 \end{pmatrix} = \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix} \left(\begin{matrix} I_3 & 0_3 \end{matrix} \right) \tilde{p}^c$$

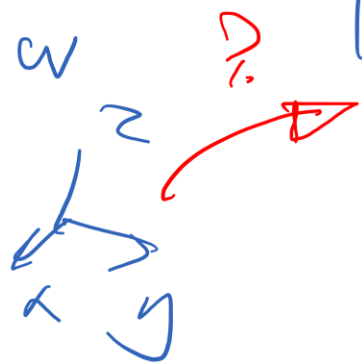
$$P_z^c \begin{pmatrix} s_m & s_a & 0_n \\ 0 & s_y & 0_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v_m^c \\ u_m^c \\ 1 \end{pmatrix} = \begin{pmatrix} s_m & s_a & 0_n \\ 0 & s_y & 0_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P_z^c \begin{pmatrix} u^T \\ v^T \\ 1 \end{pmatrix} = \begin{pmatrix} s_m f & s_a f & 0_n \\ 0 & s_y f & 0_y \\ 0 & 0 & 1 \end{pmatrix} \left(\begin{matrix} I_3 & 0_3 \end{matrix} \right) \tilde{p}^c$$

$$P^C = \underbrace{\begin{pmatrix} R_w^C & t_w^C \end{pmatrix}}_{T_w^C} P^W$$

$T_w^C \Rightarrow$ Camera extrinsic

$K \Rightarrow$ Camera Intrinsic



$$\underbrace{P_z^C \begin{pmatrix} u^T & I \\ v^T & I \\ 1 \end{pmatrix}}_{\Pi} = \underbrace{K \Pi_0 T_w^C}_{\Pi} \tilde{P}^W$$

Π : Projection matrix